
CORRIGIBILITY AS A STRUCTURAL PRECONDITION FOR DIGITAL PUBLIC INFRASTRUCTURE: A CYBERNETIC FRAMEWORK

Anivar A Aravind¹

¹Independent Researcher

January 2026

ABSTRACT

Digital Public Infrastructure (DPI) operates at the scale of populations. Systems governing identity, payment, and data exchange effectively constitute a new administrative state. While international definitions from the G20 and United Nations articulate normative aspirations (that systems *should* be open, inclusive, and safe), they largely lack the structural requirements to verify these attributes.

This paper proposes a formal evaluation framework based on **corrigibility**: the structural capacity of those affected by a system to detect error, signal harm, and trigger correction. We synthesize necessary conditions from three traditions: cybernetics (Ashby’s Law of Requisite Variety), commons governance (Ostrom’s design principles), and free software (Stallman’s four freedoms). The resulting framework identifies five non-negotiable properties: **Reversibility, Inspectability, Verification, Constraint, and Reproduction**. We demonstrate that a failure in any single dimension breaks the feedback loop required for system stability. Furthermore, we extend this framework to learned systems (AI) and establish evidentiary standards for distinguishing authentic governance from performative compliance. The framework is released under CC0 public domain.

1 Introduction

Digital Public Infrastructure is not merely a collection of software stacks; it is the encoding of political arrangements into technical artifacts [Winner, 1980]. Access to essential survival services (food distribution, banking, healthcare, mobility) is increasingly mediated through these systems. Because these systems apply fixed rules to the infinite variety of human life, they will inevitably misclassify, exclude, or fail specific users. This is not a malfunction; it is a mathematical certainty when finite rule sets encounter human diversity.

The critical question for a digital polity is not whether errors occur, but whether the architecture permits those affected to **detect, correct, and reverse** them before harm becomes permanent.

Current policy discourse defines DPI through aspirational language. The G20 New Delhi Declaration (2023) and the UN/UNDP framework [UNDP, 2024] describe systems that “should be secure” or “can be built on open standards.” These definitions exclude almost nothing. They articulate intent, not condition. Under such broad definitions, a system that traps users in a coercive loop can be labeled “public infrastructure” simply because it operates at scale.

This paper defines corrigibility not as a moral preference, but as a structural property essential for stability. The central argument is that **unverifiable constraint is structurally indistinguishable from absolute operator control**. Therefore, public legitimacy requires more than policy assertions; it requires a cryptographic chain of custody and a verifiable capacity for correction.

Contributions. This paper makes the following contributions:

*Corresponding author: anivar.net · ping@anivar.net · ORCID: [0009-0009-8995-0005](https://orcid.org/0009-0009-8995-0005)

1. **Formal Framework:** This paper formalizes DPI corrigibility through five structural tests (EXIT, CODE, AUDIT, GOVERN, FORK) derived from cybernetic control theory, commons governance, and free software principles.
2. **Stability Proof:** A control-theoretic proof demonstrates that partial corrigibility is equivalent to open-loop instability, explaining why systems that fail any single test accumulate uncorrectable harm.
3. **Empirical Application:** The framework is applied to national identity systems (Aadhaar), payment infrastructure (UPI), and learned systems (AI), demonstrating systematic evaluation of real-world DPI.
4. **Extension to AI:** The framework extends to stochastic learned systems, identifying the Temporal Variety Gap as a fundamental challenge and proposing verification methods for non-deterministic infrastructure.
5. **Machine-Readable Schema:** We provide JSON schemas enabling automated corrigibility assessment and standardized infrastructure disclosure.

Paper Structure. Section 2 diagnoses why current DPI definitions fail. Section 3 establishes the theoretical foundations from cybernetics, commons governance, and free software. Section 4 derives the five structural tests. Section 5 analyzes temporal dynamics and the correction velocity inequality. Section 6 extends the framework to AI systems. Section 7 provides empirical evaluation of existing infrastructure. Section 8 addresses implications, including essential services and agentic scaling. The appendices formalize the control-theoretic proofs and provide machine-readable schemas for automated assessment.

Key Terms.

Corrigibility

The structural capacity of those affected by a system to detect error, signal harm, and trigger correction. Not a moral preference but a stability requirement.

Requisite Variety

Ashby's Law: a controller must have at least as many states as the system it regulates. Applied to DPI: governance mechanisms must match population diversity.

Learned Systems

Infrastructure where behavior is determined by parameters learned from data (AI/ML) rather than explicit code logic.

EXIT

Test 1: Reversibility of participation without prohibitive penalty.

CODE

Test 2: Inspectability of system logic by affected parties.

AUDIT

Test 3: Independent verification of correct operation.

GOVERN

Test 4: Binding constraint that users can enforce, not advisory.

FORK

Test 5: Capacity to reproduce infrastructure independently.

Correction Velocity

Rate at which errors can be corrected; must exceed error accumulation rate for stability.

GDoS

Governance Denial of Service: when agent interaction volume exceeds governance capacity.

Open-washing

Claiming openness through partial compliance while retaining structural control.

DID-washing

Using decentralized identifiers while maintaining centralized governance.

Human Friction Buffer

The hidden subsidy where humans absorb system failures through delay and workaround, masking architectural incorrigibility.

2 The Problem: Definitions Without Structure

The accepted definitions of Digital Public Infrastructure share a common defect: they describe desired outcomes without specifying the architectural preconditions required to achieve them. This section dissects the authoritative definitions to demonstrate their structural inadequacy.

2.1 The Definitional Landscape

The G20 New Delhi Leaders' Declaration (2023) established the international consensus:

“Digital Public Infrastructure (DPI) is a set of shared digital systems that should be secure, interoperable, built on open standards, enabling delivery of services at population scale” [G20, 2023].

The UNDP framework [UNDP, 2024] elaborates:

“DPI refers to digital solutions that enable basic functions essential for public and private service delivery, including digital identity, payments, and data exchange. These solutions can be built as open-source, with open standards and specifications, and should incorporate principles of inclusion, security, and privacy by design.”

The World Bank's identification principles add:

“[Identity systems should be] robust, inclusive and accountable...responsive to the needs of various users and enrollment contexts...designed to allow for federation, interoperability, and data portability.”

2.2 Anatomizing the Aspirations

These definitions deploy a consistent vocabulary: *secure, interoperable, open, inclusive, accountable*. Examined structurally, each term collapses under scrutiny.

“Secure.” Secure for whom? A system can be cryptographically robust against external attackers while remaining structurally impervious to those it governs. The CIDR (Central Identities Data Repository) in Aadhaar is “secure” in the sense that it resists breaches. It is not secure in the sense that citizens can verify its operation or contest its determinations. Security without accountability is a fortified monopoly.

“Interoperable.” Interoperable with what? A system that speaks standard protocols to upstream services while maintaining proprietary control over downstream users achieves interoperability for operators, not subjects. UPI's API specifications are interoperable; citizens cannot interoperate with NPCI's switching logic. The interface is public; the machine is private.

“Open Standards.” Open standards are not open execution. HTML is an open standard; Chrome's rendering engine is proprietary. A browser speaks a public language while its internal logic remains invisible. Similarly, a payment rail can implement open APIs while its routing algorithms, fee structures, and dispute resolution mechanisms remain black boxes. Publishing a specification does not make a system inspectable.

“Inclusive.” Inclusive under what conditions? A system that enrolls everyone achieves enrollment inclusivity. But enrollment is the beginning, not the end. A system that enrolls everyone while providing no mechanism for contestation or exit achieves inclusive capture. The question is not “Can everyone get in?” but “Can anyone get out?” Mandatory enrollment with no exit is not inclusion; it is mandatory lock-in.

“Accountable.” Accountable to whom, through what mechanism, with what enforcement? The definitions invoke accountability as an adjective, not a structure. They specify no verification procedure, no audit right, no correction channel. Accountability without mechanism is rhetoric.

2.3 The Four Structural Gaps

The definitional inadequacy manifests as four distinct capture vectors:

First, Open Standards are not Open Execution. A proprietary system can speak a standard language while remaining entirely opaque in its internal logic. (Addressed by the **CODE** test, Section 4.)

Second, Legal Frameworks do not guarantee Technical Rights. A system that requires a court order to correct a database error effectively denies correction to the majority of its users. Legal remedy operates on bureaucratic time; infrastructure operates on digital time. (Addressed by the correction velocity inequality, Section 5.)

Third, Aspiration is not Compliance. Definitions that rely on what a system “can” do or “should” be offer no resistance to authoritarian drift. Modal language creates no enforceable constraint. Every term in the G20 definition is advisory (“should be secure,” “can be built on open standards”). Advisory language excludes nothing. (Addressed by the **GOVERN** test, Section 4.)

Fourth, Scale is not Accountability. A system that serves a billion users achieves universality, but universal deployment does not confer public legitimacy. Population-scale operation may simply mean population-scale capture. The “public” in such definitions refers to the number of subjects, not the structure of control. (Addressed by the essential services problem, Section 7.6.)

2.4 The Core Thesis

Without hard structural requirements, “Public” becomes a mere descriptor of scale rather than a mode of accountability. Just as classification systems become invisible as they gain acceptance [Bowker and Star, 1999], incorrigible infrastructure recedes into the background, becoming unchallengeable precisely as it becomes essential.

Remark 1 (The Definitional Problem). *Current DPI policy specifies **intent**, not **conditions**. It articulates what systems should achieve without specifying what they must structurally provide. Under such definitions, a system that traps users in a coercive loop can be labeled “public infrastructure” simply because it operates at scale. The framework proposed in this paper converts aspirational adjectives into verifiable predicates. It asks not “Is this system described as open?” but “Can independent parties inspect its operation?” Not “Is this system intended to be accountable?” but “Can those it affects trigger correction?”*

This danger is acute in the transition to AI-mediated governance, where “Infrastructure” begins to make probabilistic judgments rather than deterministic records. When the system itself becomes opaque by design (learned behavior rather than written code), the gap between definitional aspiration and structural reality widens further.

2.5 Related Work

The DPI literature spans policy frameworks, technical architectures, and critical analyses, but lacks a unified structural standard for accountability.

Policy Frameworks. The G20 framework [G20, 2023] and UNDP definitions [UNDP, 2024] establish aspirational principles (openness, inclusion, interoperability) without verification criteria. The World Bank’s Identification for Development (ID4D) initiative emphasizes functional identity but does not specify architectural requirements for user exit or independent audit. These frameworks describe *what* DPI should achieve without specifying *how* to verify achievement.

Critical Analyses. Scholarship on India Stack has documented exclusion harms [Khera, 2019] and constitutional tensions [Bhatia, 2019]. However, these critiques remain case-specific rather than providing generalizable tests applicable across jurisdictions and technologies.

Control Theory and Governance. Cybernetic approaches to governance date to Ashby [Ashby, 1956] and Beer’s viable systems model, but have not been systematically applied to digital infrastructure accountability. Lessig’s “code is law” [Lessig, 2006] establishes that architecture constrains behavior, but does not specify which architectural properties are necessary for democratic legitimacy.

Software Freedom. The free software tradition [Stallman, 2002] provides operational tests (inspect, modify, redistribute) but focuses on code artifacts rather than infrastructure systems that include data, compute, and network effects.

Gap and Contribution. No existing framework provides falsifiable, technology-agnostic tests for infrastructure accountability that (1) derive from formal stability theory, (2) apply equally to deterministic and learned systems, and (3) yield machine-verifiable assessments. This paper addresses that gap by synthesizing cybernetics, commons governance, and software freedom into five structural tests with formal proofs and empirical application.

3 Theoretical Foundations

This framework does not invent new ethical categories. Instead, it triangulates a single structural requirement from three independent intellectual traditions. Whether viewed through physics, political economy, or code, the stability of a system depends on the capacity of its subjects to provide corrective feedback.

3.1 Cybernetics and Requisite Variety

In cybernetics, the relationship between a controller and its environment is governed by Ashby’s Law of Requisite Variety [Ashby, 1956]. The law states that for a system to be stable, the number of control states must meet or exceed the number of environmental states:

$$V(\text{controller}) \geq V(\text{disturbance}) \quad (1)$$

In DPI, the “disturbance” is the boundless diversity of the human population. No pre-programmed rule set can match this variety. Stability, therefore, relies on feedback channels that transmit error signals from the population back into the system’s behavior. If these channels (specifically the ability to exit or correct) are blocked, the controller’s variety effectively collapses. The system loses the capacity to regulate its own impact, leading to accumulated errors that manifest as either instability or systematic exclusion.

Human variety includes technical and social edge cases that deterministic rule sets cannot exhaustively enumerate: differences in biometric capture conditions (e.g., manual laborers with worn fingerprints, elderly individuals with fading iris patterns), non-standard household and family structures that diverge from canonical registries, multilingual and script variance that invalidate literal string matches, and intermittent or low-bandwidth contexts where real-time digital verification is impossible. These concrete phenomena instantiate Ashby’s theoretical argument: controllers lacking channels for diverse corrective signals will systematically misclassify and exclude.

3.2 The Commons and Constitutional Constraint

Elinor Ostrom’s work on commons governance [Ostrom, 1990] establishes that sustainable systems require monitors who are accountable to the users and collective-choice arrangements that allow users to modify the rules. A system in which rule-making is the exclusive preserve of the operator acts as an enclosure, not a commons.

Critically, Ostrom identifies that governance is not merely about access; it is about graduated sanctions and low-cost conflict resolution. Applied to digital infrastructure, this implies that users must possess **Constitutive Power**: the ability to set binding limits on the system’s behavior. If a system’s constraints are merely advisory or can be overridden by the operator at will, the system is fundamentally ungoverned.

3.3 Free Software and the Right to Reproduce

The Free Software movement [Stallman, 2002] contributes the mechanical means of enforcement. Stallman’s “Four Freedoms” are often misread as a philosophy of sharing, but they are functionally a philosophy of power. The freedom to study (Inspectability) ensures power is visible; the freedoms to modify and distribute (Reproduction) ensure that power is not a monopoly.

This tradition clarifies that openness is not a passive property (reading code) but an active one (forking infrastructure). Transparency without the power to act on it is functionally inert. The ability to reproduce the system independent of its original steward is the only ultimate check against operator capture.

4 The Structural Conditions of Corrigibility

This framework defines corrigibility as the existence of specific mechanical capabilities that allow the subjects of a system to maintain control over it. Digital Public Infrastructure cannot rely on intent or policy to guarantee this property; it requires a specific architecture.

These capabilities form a hierarchy of defense. If a system lacks any single capability, the feedback loop breaks. Such a system becomes structurally indistinguishable from a monopoly exercising absolute control.

4.1 Test 1: EXIT (Reversibility of Participation)

Definition 1 (EXIT). A system S satisfies **Reversibility of Participation** if, for any user in state $s \in S$, there exists a transition to a non-participatory state s_0 such that the penalty $\pi(s \rightarrow s_0)$ is strictly bounded: $\pi(s \rightarrow s_0) < \tau_{\text{exit}}$, where τ_{exit} is a threshold below which the penalty does not constitute material exclusion from essential services.

The foundational condition of any corrigible system is the **Reversibility of Participation**. Irrevocable consent functions structurally as mandatory lock-in. For an infrastructure to operate as a public utility rather than a coercive instrument, users must possess the capacity to withdraw from it without incurring disproportionate penalty or material exclusion.

In practice, many systems operate as a ratchet. They are easy to enter but functional imperatives make them impossible to leave. **Aadhaar** exemplifies this failure: opting out results in exclusion from banking, food rations, and mobile connectivity. When a system becomes a prerequisite for existence, refusal results in a disproportionate survival penalty rather than functioning as feedback. **UPI** partially mitigates this because cash remains legal tender, but network effects increasingly marginalize non-participants. **Linux**, by contrast, imposes no survival penalty: a user can migrate to BSD or Windows without losing access to essential services.

From a cybernetic perspective, EXIT constitutes the primary negative feedback loop. Blocking this channel silences the user’s error signal. This blockade overloads the political layer with demands that the technical layer has suppressed.

This dynamic was formalized by Hirschman [Hirschman, 1970] in his analysis of organizational decline. Hirschman distinguished two correction mechanisms: exit (withdrawal from participation) and voice (attempts to change the system from within). His central finding is that these mechanisms exist in productive tension, not as substitutes.

Exit, in Hirschman’s formulation, is “neat,” “impersonal,” and “indirect”: one either exits or one does not. This binary quality makes exit a high-bandwidth error signal. When users leave, the system receives unambiguous information that something has failed. Voice, by contrast, is costly and conditioned on the influence and bargaining power that participants can bring to bear. Voice is gradual, political, and requires sustained effort.

Critically, Hirschman demonstrated that exit without voice produces abandonment rather than correction: quality-conscious users flee rather than fight. But voice without exit produces capture: complaints become cosmetic (see Section 5.3.1) because the system faces no credible threat of user departure. The effectiveness of voice depends on the credibility of exit.

This framework illuminates why mandatory enrollment is structurally pathological. By eliminating exit, a system does not merely suppress one feedback channel. It degrades voice itself. When citizens cannot leave, their complaints lack credibility. The operator can absorb infinite grievance without modifying system behavior ($\partial S / \partial G = 0$). Hirschman’s insight formalizes what cybernetics predicts: blocking the error signal does not eliminate the error; it eliminates the system’s capacity to respond.

4.2 Test 2: CODE (Inspectability of Logic)

Definition 2 (CODE). A system S satisfies **Inspectability of Logic** if, for any decision function $f : X \rightarrow Y$ that determines user outcomes, there exists a publicly accessible artifact A_f such that A_f fully specifies the computation of $f(x)$ for all inputs $x \in X$. For learned systems, A_f includes training data provenance, model architecture, and alignment procedures.

Users who cannot leave a system must possess the ability to inspect it. **Inspectability of Logic** requires that the rules determining outcomes are visible to those they govern. Power in a digital system resides in execution. Lessig [Lessig, 2006] established that software architecture functions as regulation: code “determines what people can or cannot do in the first place,” unlike legal regulation which sets rules for behavior and retroactively punishes non-compliance. This is regulation without appeal. If the execution path remains hidden, that power is absolute; it is not merely unaccountable but structurally unobservable to those it governs.

Transparency specifications or open standards are insufficient if the executing artifact remains a black box. **Aadhaar’s** CIDR (Central Identities Data Repository) is proprietary: the matching algorithm that determines identity cannot be examined. **UPI** publishes API specifications, but the NPCI switching logic remains closed. **Let’s Encrypt** demonstrates the alternative: its CA software is fully open source, allowing anyone to audit the certificate issuance logic that secures web traffic. For learned systems, the challenge intensifies. **Llama** publishes inference weights but withholds training data composition and RLHF preferences. This is spec-as-source: you can run the model but cannot understand why it produces specific outputs.

4.3 Test 3: AUDIT (Independent Verification)

Definition 3 (AUDIT). A system S satisfies **Independent Verification** if any external party P , without requiring operator authorization, can measure the system’s error rate ϵ_S , verify compliance with stated specifications, and document failure modes in production environments. Formally: $\forall P : \text{Access}(P, \epsilon_S) = \text{true}$ without $\text{Authorize}(\text{Operator}, P)$.

While inspection enables understanding, **Independent Verification** enables truth. This functions as the sensor in the cybernetic feedback loop. A system is verifiable only if external, permissionless actors can test its behavior, measure its error rates, and document its failure modes within production environments.

Remark 2 (The Auditability Distinction). The framework does not equate auditability with direct participation. *AUDIT* $\neq$ “everyone audits”; *AUDIT* = “no one can prevent auditing.” **Journalism analogy**: Not everyone investigates corruption. Democracy still depends on the possibility of investigation. The problem today is not that auditors exist; operators can legally block them. The tradeoff is contestable expertise over unchallengeable authority—a defensible democratic position.

Incorrigible systems typically capture this function. **UPI** permits RBI regulatory audits, but transaction-level verification requires NPCI authorization; independent researchers cannot measure failure rates or dispute resolution times. **Open-weights AI models** (Llama, Qwen, DeepSeek) selectively publish red-teaming results while blocking independent adversarial testing at scale. **Bitcoin** provides the counter-model: every transaction is independently verifiable by any node, and consensus is cryptographic proof. Similarly, **Let’s Encrypt** publishes Certificate Transparency logs, making every issued certificate publicly auditable in real time.

The structural function of opacity is sensor failure. A system whose error rates cannot be independently measured cannot be regulated. The controller receives no signal about divergence from intended behavior. Under Ashby’s Law, such a system will drift until external forces impose correction. Those forces, operating outside the technical loop, will impose correction through channels the system cannot anticipate or absorb. Pasquale [2015] describes this as a “one-way mirror” relationship, but the cybernetic translation is precise: asymmetric observability breaks the feedback loop at the sensing stage.

4.4 Test 4: GOVERN (Constitutive Constraint)

Definition 4 (GOVERN). A system S satisfies **Constitutive Constraint** if there exists a governance function $G : \text{Rules} \rightarrow \text{Rules}'$ such that (1) G is accessible to affected parties, not solely the operator; (2) G is binding—the operator cannot override G ’s output; and (3) there exists a cryptographic or legal chain of custody linking system behavior to G ’s constraints.

Verification identifies the error, while **Constitutive Constraint** enforces the correction. This framework distinguishes structural governance from administrative functions like consultation, multi-stakeholder meetings, or feedback forms.

Governance exists only when binding constraints limit system behavior in ways the operator cannot override. **Aadhaar** is governed by UIDAI fiat: the same entity that operates the system sets its rules. **UPI** has NPCI governance, but NPCI is majority-owned by participating banks, creating structural conflicts. **Let’s Encrypt** shows what binding constraint looks like: the ISRG charter legally binds the organization to its public benefit mission, with community board representation. **Bitcoin**’s BIP process demonstrates governance through proven fork history; contentious changes have been rejected by the network, proving the community can override developer intent. **PostgreSQL**’s contributor agreement prevents any single entity from capturing the project.

4.4.1 The Post-Execution Fallacy

A critical failure in modern policy involves relying on exogenous or post-hoc remedies to substitute for structural constraints. Mechanisms such as courts, ombudsmen, and grievance redressal officers operate on “bureaucratic time,” measured in months or years. Infrastructure operates on “digital time,” measured in milliseconds. A system that executes a harmful action, such as wrongful deletion of a beneficiary, and relies on a court order to reverse it six months later is structurally ungoverned for that duration. True governance must function within the system’s execution loop to block prohibited states before they manifest.

Consequently, claims of governance must be evidentiary rather than declarative. They require a signed cryptographic chain of custody linking the operator’s authority to an external, non-overrideable instrument.

4.4.2 Strategic Openness vs Structural Accountability

Open artifacts do not guarantee open governance. Both market-gardener and state-architect doctrines deploy openness instrumentally: releasing code or weights for adoption while retaining control over training, direction, and future

development. Llama’s weights are public; Meta controls alignment decisions. Android’s AOSP is open; Google controls Play Services. This pattern (asymmetric openness) releases the artifact while closing the correction loop.

The GOVERN test requires bidirectional accountability: not merely that citizens can observe the system, but that they can correct it. Openness can be weaponized. Strategic openness maximizes visibility into users while minimizing user power over the system. This is the inverse of corrigibility: surveillance, not governance. True corrigibility requires the correction loop be open, not merely the artifacts. A system that publishes its code while reserving all governance decisions to the operator has satisfied CODE but failed GOVERN.

A related confusion conflates governing one’s own data in a system with governing the system itself. Data protection frameworks (GDPR, CCPA) grant users rights to access, correct, delete, and port their data. These rights are necessary but insufficient. They allow users to manage information within rules the operator defines. They do not allow users to change those rules. Privacy dashboards that grant data control while withholding system control are structural theatre. The GOVERN test asks: can affected parties change the rules, not merely operate within them?

4.5 Test 5: FORK (Independent Reproduction)

Definition 5 (FORK). *A system S satisfies **Independent Reproduction** if an independent party can instantiate a functionally equivalent system S' such that (1) no legal prohibition prevents S' ’s deployment; (2) the artifacts required for reproduction (code, schemas, protocols) are publicly available; and (3) user state U_S is portable: $U_S \rightarrow U_{S'}$. Natural barriers (capital, network effects) do not disqualify FORK; only constructed barriers (legal prohibition, regulatory monopoly) do.*

The ultimate check on power is the ability to recreate it. **Independent Reproduction** determines whether the ecosystem allows the infrastructure itself to be separated from its current steward. This requirement distinguishes structural resilience from simple market competition.

Remark 3 (Credible Replaceability). *The paper does not define FORK as “routine divergence in production.” It defines FORK as **credible replaceability in extremis**. **Constitutional analogy:** The US Constitution is forkable (amendable, secession historically possible). That does not mean every county runs a different constitution. The possibility disciplines the center. **The stable equilibrium:** Latent forkability + strong coordination incentives. Email works not because everyone forks SMTP, but because anyone could. **Key asymmetry:** FORK is a threat, not a preference. No one wants fragmentation. But irreversibility without exit is worse.*

The existence of competitors, such as choosing between two proprietary cloud providers, provides an “Exit” option but not a “Fork.” Shifting providers allows a user to escape a specific operator, but it forces them to abandon their history, identity, and operational logic. This action is substitution, not reproduction.

4.5.1 Natural vs Constructed Barriers

A critical distinction must be drawn between barriers to forking that are **natural** (arising from coordination costs, network effects, or capital requirements) and those that are **constructed** (arising from legal prohibition, regulatory monopoly, or deliberate technical enclosure).

- **Natural barriers:** Gmail dominates email not because forking SMTP is illegal, but because Google invested in spam filtering and UX. Anyone *can* run their own mail server. Network effects and capital create friction, but not prohibition.
- **Constructed barriers:** NPCI is the *only permitted* instant payment switch in India. The barrier is regulatory, not economic. Even with infinite capital, you cannot legally compete with UPI’s core switching function.

The FORK test fails only when constructed barriers prevent reproduction. Natural barriers reduce the *likelihood* of forking; constructed barriers eliminate its *possibility*. A system with high natural barriers but no constructed barriers (Linux, email) remains structurally corrigible. A system with low natural barriers but constructed prohibition (hypothetically: open-source software banned by law) would fail FORK despite technical simplicity.

4.5.2 State Portability as a Precondition for Fork

A critical structural requirement: code forkability is insufficient if **user state** is non-transferable. In traditional software (Linux, PostgreSQL), the artifact being forked is the system. In Digital Public Infrastructure, the artifact is merely the skeleton; the system includes the user’s credentials, transaction history, social graph, and accumulated trust relationships.

Let U_S denote the user state graph in system S , and S' a forked alternative. A fork is **functionally meaningful** only if:

$$U_S \rightarrow U_{S'} \quad (\text{state portability}) \quad (2)$$

Open code with locked data produces a hollow fork. The legal right to reproduce an identity system is meaningless if citizens cannot port their credentials, attestations, and service eligibility to the new instance.

Switching Cost (σ). We operationalize the practical barrier to exit and fork through a **switching cost** scalar:

$$\sigma(S \rightarrow S') = c_{\text{data}} + c_{\text{downtime}} + c_{\text{reprovisioning}} + c_{\text{network}} + c_{\text{legal}} \quad (3)$$

where each component is normalized to $[0, 1]$:

- c_{data} : cost of exporting and reformatting user data
- c_{downtime} : service interruption during transition
- $c_{\text{reprovisioning}}$: re-establishing credentials and trust relationships
- c_{network} : loss of social connections and network effects
- c_{legal} : regulatory friction and contractual barriers

Worked Example: Aadhaar vs UPI. Consider switching from Aadhaar-linked services to hypothetical alternatives:

System	c_{data}	c_{down}	c_{reprov}	c_{net}	c_{legal}	σ
Aadhaar $\rightarrow$ Alt ID	1.0	1.0	1.0	0.8	1.0	4.8
UPI $\rightarrow$ Cash/Card	0.2	0.1	0.3	0.4	0.0	1.0
UPI $\rightarrow$ Alt Payment	0.3	0.2	0.4	0.5	0.1	1.5

Table 1: Illustrative switching cost estimates. Aadhaar exhibits near-maximal σ because no data export exists ($c_{\text{data}} = 1$), service denial during transition is total ($c_{\text{down}} = 1$), and legal mandate forecloses alternatives ($c_{\text{legal}} = 1$). UPI retains lower σ because cash and cards remain legal alternatives.

With $\Sigma^* = 2.0$ as a policy threshold, Aadhaar ($\sigma = 4.8$) categorically fails EXIT/FORK while UPI ($\sigma = 1.0\text{--}1.5$) passes conditionally. The calculation reveals that Aadhaar’s incorrigibility is not merely a governance failure but a mathematical certainty given its architectural constraints.

The Portability Threshold. EXIT and FORK functionally fail when switching cost to all plausible alternatives exceeds a policy threshold:

$$\min_{S' \in \mathcal{A}} \sigma(S \rightarrow S') > \Sigma^* \implies \text{EXIT/FORK fail} \quad (4)$$

where $\mathcal{A}$ is the set of available alternatives and Σ^* is a calibrated threshold (e.g., > 0.2 of annual disposable income or > 30 days service disruption).

Technical Requirements for Portability. To ensure $\sigma < \Sigma^*$, DPI systems must implement:

1. **Standardized export bundle:** user-owned, signed, encrypted archive containing identity claims, transaction history, and verifiable credentials (format: JSON-LD with signed manifests)
2. **Interoperability API:** “ingest” endpoint specification to accept ported bundles with defined mapping rules and time-to-accept guarantees (e.g., 72 hours)
3. **Statutory acceptance guarantee:** regulation requiring public services to accept verified ported credentials if cryptographic lineage checks pass

This is the lesson of PSD2 (Payment Services Directive 2) in banking: portability mandates reduced c_{data} and c_{legal} , making bank-switching economically viable. Similar mandates are required for identity and data infrastructure.

A system passes the FORK test only if the community possesses the legal rights, technical artifacts, economic means, **and state portability guarantees** to spawn a functionally equivalent instance. **Aadhaar** fails completely: no entity outside UIDAI can reproduce the identity infrastructure. **OpenSearch** demonstrates what successful forking looks like. When Elastic changed its license, AWS forked Elasticsearch and the community continued development independently. **Linux** has been forked thousands of times (Android, Chrome OS, countless distributions). **Llama** presents partial forkability: anyone can run the weights, but reproducing the training requires resources only a handful of companies possess. If Meta abandoned Llama tomorrow, the community could not recreate it.

System	EXIT	FORK	Structural Reality
Cloudflare	PASS (Market)	FAIL	Can switch to Fastly (Exit), cannot reproduce (Proprietary)
AWS	PASS (Market)	FAIL	Can migrate to Azure, logic/state locked (Ecosystem capture)
Aadhaar	FAIL (Mandatory)	FAIL	Cannot refuse (Legal), cannot reproduce (Total capture)
Linux	PASS	PASS	Can leave, can reproduce/fork (Corrigible infrastructure)
Llama	PASS	PARTIAL	Can refuse, can run weights, cannot reproduce training

Table 2: Differentiation between Exit (Substitution) and Fork (Reproduction)

4.6 The Topology of Necessity

Five tests describe the anatomy of a stable control loop rather than a random assortment of normative virtues (see Figure 1).

- **EXIT** provides the Error Signal.
- **AUDIT** acts as the Sensor.
- **CODE** ensures Transmission Clarity.
- **GOVERN** serves as the Actuator.
- **FORK** provides Selection Pressure over the entire loop.

Failure in any single dimension breaks the control loop at a specific physical point. The result is not partial infrastructure but **Open Loop Control**. An open-loop system executes directives without the capacity to sense deviation, interpret error, or apply corrective force.

Under conditions of human variety and environmental change, open-loop control is structurally unstable. Error accumulates without bound. Misalignment compounds. The system drifts toward failure.

Corrigibility is the structural precondition that closes the loop. Only when all five tests are satisfied does a system possess a complete feedback topology capable of detecting divergence, transmitting intelligible signals, executing correction, and replacing failed control logic when necessary. Without this closure, stability is mathematically and operationally impossible.

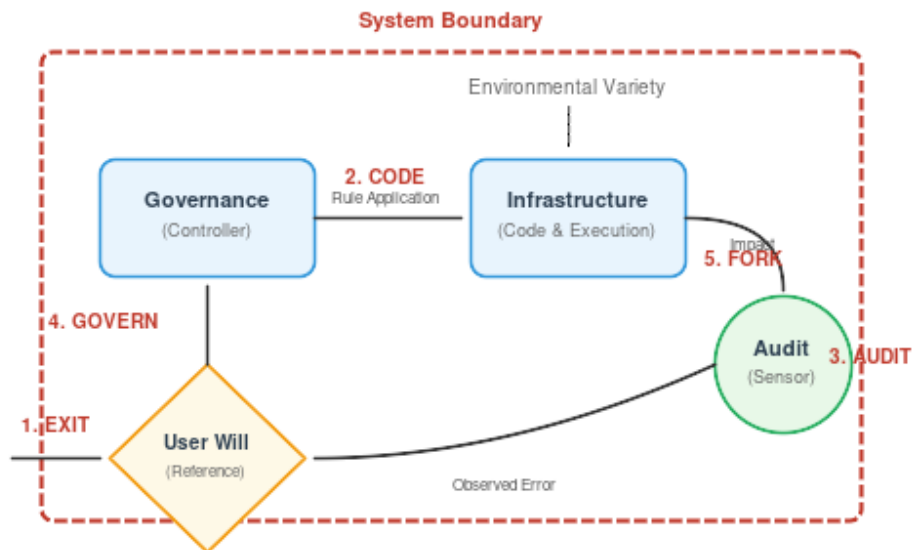


Figure 1: **Topology of Necessity**. Each corrigibility test corresponds to a breakage point in the feedback control loop.

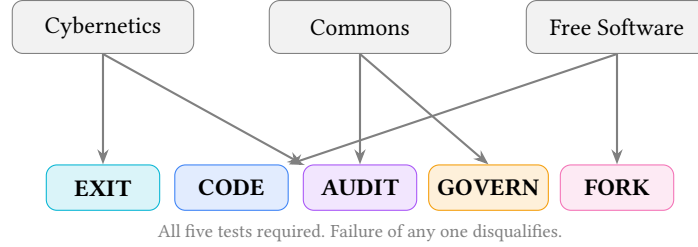


Figure 2: Five corrigibility tests derived from three independent theoretical traditions.

4.7 Structural Symmetry of Power

The five tests, while derived from distinct intellectual traditions, share a common organizing principle. Each enforces a constraint on power concentration at a different layer of the system. Together, they form a single power invariant:

Layer	Test	Function (Cybernetic)
Individual	EXIT	Reversibility of participation
Logic	CODE	Inspectability of rules
Logic	AUDIT	Verifiability of error
System	GOVERN	Constraint on controller variance
Ecosystem	FORK	Selection pressure via reproduction

Each test enforces the same principle: *No actor exercising power may be the sole judge of its continuation, scope, or correction.*

A system without EXIT is coercive. A system without GOVERN is arbitrary. A system without FORK is captive.

4.8 Layer-Decomposed Evaluation

Complex infrastructure frequently stratifies into distinct functional layers, each with independent governance, transparency, and reproducibility properties. The five tests must be applied to each layer separately when:

1. Different entities control different layers (protocol versus implementation versus trust framework)
2. Transparency varies across layers (open protocol, closed switching logic)
3. Forkability differs by layer (code is forkable, network effects are not)

4.8.1 Propagation Rule

A system that passes a test at one layer but fails at another does not achieve “partial” compliance. It fails the test. The evaluation proceeds from the layer closest to user impact upward; failure at any layer propagates to the whole.

Formally: Let $L_1, L_2, \dots, L_n$ be the layers of a system, ordered by proximity to user impact. For test T :

$$T_{\text{system}} = \bigwedge_{i=1}^n T(L_i) \quad (5)$$

The system passes T if and only if every layer passes T . The full corrigibility determination extends this across all five tests:

$$\text{Corrigible}(S) = \bigwedge_{t \in \mathcal{T}} \bigwedge_{i=1}^n T_t(L_i) \quad \text{where } \mathcal{T} = \{\text{EXIT}, \text{CODE}, \text{AUDIT}, \text{GOVERN}, \text{FORK}\} \quad (6)$$

This double conjunction formalizes the “weakest link” principle: failure of any test at any layer propagates to the system determination.

Layer	Example Components	Typical Failure Mode
Protocol	W3C DID, VC Data Model	Often passes CODE
Implementation	Specific resolver, wallet software	May fail CODE (proprietary)
Credential	Issuer policies, attribute schemas	Often fails GOVERN (unilateral issuer)
Trust	Relying party acceptance, trusted lists	Often fails FORK (cannot reproduce acceptance)

Table 3: Identity infrastructure layer decomposition

4.8.2 Application to Identity Infrastructure

The UPI payment rail exhibits similar layering: the protocol specification (NPCI-published) passes CODE at the protocol layer, but the switching logic (NPCI-operated) fails CODE at the implementation layer. Since the implementation layer is closer to user impact, the system fails CODE.

4.8.3 The Weakest-Layer Principle

This propagation rule has a counterintuitive but structurally necessary consequence: a system’s corrigibility status equals its weakest layer across any test dimension. Strength at one layer cannot compensate for failure at another.

This principle prevents a common evasion: claiming corrigibility by publishing peripheral artifacts while retaining control over the layers that determine outcomes. Open-washing (Section 7.4) typically exploits layer confusion. Releasing SDKs (passing CODE at the interface layer) while keeping core logic proprietary (failing CODE at the execution layer) does not produce partial corrigibility. It produces total failure at the execution layer, which propagates to the system determination.

5 The Dynamics of Incorrighibility

The five tests define the static architecture of a corrigible system. However, infrastructure exists in time. When we apply these structural requirements to dynamic systems, three fundamental implications emerge regarding stability, energy conservation, and the rule of law. These principles explain why centralized grievance redressal mechanisms (the standard answer of bureaucracies to error) are mathematically insufficient to guarantee stability.

5.1 The Harm Accumulation Principle

A critical clarification is required regarding the framework’s claims about incorrigible systems:

Principle (Harm Accumulation): Incorrigible systems accumulate harm until correction becomes catastrophic rather than incremental. The framework does NOT claim “incorrigibility leads to collapse.” It claims that **corrigibility is a precondition for a system to be meaningfully public.**

This is ontological classification, not historical prediction. North Korea is stable, incorrigible, and not public in any meaningful sense, confirming rather than refuting the framework. Tyrannies can persist indefinitely while crushing their subjects. The system stabilizes; the humans do not.

Remark 4 (The Core Claim). *Corrigibility is not a guarantee of justice. It is a guarantee that **injustice is contestable**.*

A prison can be stable. A prison can even be efficient. But it is not public. The framework draws this line.

5.2 The Correction Velocity Inequality

Stability in a cybernetic system is not merely a function of whether a correction path *exists*, but of its *latency* relative to error generation. A correction mechanism that is theoretically available (e.g., a court case or a legislative amendment) but slower than the rate of divergence creates a runaway failure state.

The **Temporal Stability Condition** states: A system is corrigible only if the rate of effective correction (C) exceeds the rate of error accumulation (E) caused by environmental variety.

$$\frac{d}{dt}C(t) > \frac{d}{dt}E(t) \quad (7)$$

However, correction velocity has both a **lower bound** (preventing harm accumulation) and an **upper bound** (preventing manipulation):

$$v_{\min} < v_{\text{correction}} < v_{\max} \quad (8)$$

Remark 5 (Velocity Bounds). *Constitutional democracy invented latency: bicameralism, judicial review, amendment supermajorities. Friction is not failure; friction is how minorities survive majorities. The gap between accusation and punishment is where due process lives. **Instantaneous algorithmic correction bypasses necessary due process.** The framework does not argue for automated, instant, or algorithmic governance. It argues for **availability of correction channels, not their speed.** Nothing in EXIT, GOVERN, or AUDIT requires immediacy. They require possibility without permission. Courts are a designed correction channel. They satisfy GOVERN (binding authority), AUDIT (independent review), and EXIT (injunctions prevent harm). **The framework criticizes absence, not latency.***

$E(t)$ is driven by environmental volatility: identity fraud tactics, software exploits, pandemics, or economic shocks. These phenomena tend to grow exponentially or appear in high-velocity impulse shocks. In contrast, centralized governance operates on **bureaucratic time**. Legislative changes and judicial reviews are measured in months or years, while technical errors accumulate on **digital time**, measured in milliseconds. Appendix B provides a formal control-theoretic treatment of these claims; no additional assumptions beyond those stated here are introduced.

$$\frac{dC_{\text{bureaucratic}}}{dt} \ll \frac{dE_{\text{digital}}}{dt} \quad (9)$$

5.2.1 Integral Stability and Robustness Margin

The instantaneous condition $C(t) > E(t)$ is necessary but insufficient. A system may satisfy this condition momentarily while accumulating net harm over time. The **integral stability condition** captures persistence:

$$\int_{t_0}^{t_1} (C(t) - E(t)) dt \geq 0 \quad \forall t_1 > t_0 \quad (10)$$

This requires no net harm accumulation over any time window. A system that corrects errors eventually but accumulates harm during delay violates this condition.

For practical robustness against measurement noise and burst errors, the framework defines a **robustness margin** $\delta > 0$:

$$C(t) - E(t) > \delta \quad (\text{robust stability}) \quad (11)$$

Systems operating at $C(t) \approx E(t)$ are marginally stable; any perturbation tips them into harm accumulation. The margin δ represents excess correction capacity that absorbs shocks. Empirical calibration of δ requires measuring error and correction rates during incident windows (see Future Work).

This timescale mismatch is an instance of the Collingridge Dilemma [Collingridge, 1980]: a double-bind in technology governance. Early in a technology’s development, change is easy but the need for change cannot be foreseen (the information problem). Later, when impacts become clear, change has become expensive, difficult, and time-consuming because the technology is entrenched (the power problem).

Digital Public Infrastructure exhibits the Collingridge Dilemma in acute form. By the time exclusion effects are documented (starvation deaths, banking denials, welfare failures), a system may have achieved mandatory status. Correction now requires not merely technical modification but legal amendment, bureaucratic reorganization, and political will. The system’s success in achieving scale has made it resistant to correction.

Corrigibility is the structural response to this dilemma: build the correction mechanisms before the need for correction is apparent. EXIT, CODE, AUDIT, GOVERN, and FORK are not remedies applied after failure; they are architectural features that must be present from deployment. A system without these features will eventually require correction but will lack the capacity to receive it.

This inequality explains the failure of the “post-execution” remedies discussed in Section 4.4. Mechanisms like ombudsmen or grievance redressal officers inherently operate at a lower bandwidth and higher latency than the systems they supervise. When the correction loop is thermodynamically too slow to manage the entropy of the environment, error accumulates in the system until it suffers catastrophic collapse. Corrigibility requires closing the

gap between error generation and correction; this is only possible if correction logic operates within the technical loop (Test 4), not outside it.

5.3 The Conservation of Correction Demand

When a system fails to correct errors due to the velocity mismatch described above, the demand for correction does not evaporate. It transforms. The principle of **Conservation of Correction Demand** states: for a given system state, the total pressure for correction is constant.

$$F_{\text{designed}} + F_{\text{emergent}} = K \quad (12)$$

Here, F_{designed} represents the flow of correction through designed channels, principally **EXIT** (reducing participation) and **GOVERN** (altering rules). When these channels are structurally blocked or throttled by bureaucracy, the pressure shifts to F_{emergent} . Emergent correction channels take the form of protest, litigation, hacks, grey markets, and eventually, civil unrest.

System operators can choose *where* correction occurs; they cannot choose *whether* it occurs. By suppressing EXIT (via mandatory laws) and blocking GOVERN (via executive fiat), authorities force correction demand into channels that are slower, costlier, and fundamentally more destabilizing to the social order. Corrigibility acts as a pressure relief valve that converts potential volatility into system evolution.

5.3.1 Grievance Is Not Feedback

A critical distinction must be drawn between *grievance* and *governance*. Many incorrigible systems act as “roach motels” for complaints: they allow users to submit unlimited feedback (G), but the system state (S) remains invariant to that input.

$$\frac{\partial S}{\partial G} = 0 \quad (13)$$

Grievance channels that record dissatisfaction without a binding mechanism to modify execution, policy, or eligibility logic do not constitute a feedback loop. They serve an aesthetic function, mimicking responsiveness while preserving the controller’s rigidity. Under Ashby’s Law, if the controller’s response variety does not increase in response to signal, regulation has failed.

5.3.2 Active Deception

A more severe pathology occurs when systems do not merely fail to correct errors but actively misrepresent their state or capabilities. We term this **Active Deception**: the systematic production of false signals that degrade the epistemic environment for external oversight.

Active Deception takes three forms:

1. Safety-Washing. Systems that claim safety certifications, audits, or approvals that do not correspond to verifiable structural properties. A system may publish “privacy by design” documents while implementing opaque centralized logging; display compliance badges while blocking independent audit access; or cite ethics board oversight while exempting operational decisions from board review. The deception lies in the gap between representation and verifiable architecture.

2. Open-Washing. Systems that claim openness while structurally foreclosing the rights that openness implies. Publishing inference code while withholding training data, weights, or RLHF preferences constitutes open-washing: it mimics CODE compliance while preventing meaningful reproduction (FORK). The term “open” has been systematically degraded through this practice, requiring the framework to specify *structural* rather than *nominal* criteria.

3. Audit Theatre. Systems that permit nominal inspection while structurally preventing the inspector from detecting errors. This includes: providing read access to sanitized logs rather than raw execution traces; permitting audits only at pre-announced times; or granting access to components that do not determine actual behavior. The auditor observes a Potemkin system; the production system operates under different logic.

These pathologies share a common structure: they corrupt the feedback channel by injecting false negatives. When external observers receive signals indicating “compliant,” “safe,” or “open,” they rationally reduce oversight intensity. The deception is not merely a failure to inform but an active degradation of the correction system’s sensor function.

Under the framework’s cybernetic model, Active Deception constitutes a **sensor attack**: the deliberate corruption of the AUDIT channel to prevent the control loop from detecting deviation. Unlike passive incorrigibility (which merely blocks correction), Active Deception induces false confidence that accelerates divergence.

5.4 The Structural Preconditions of Law

These dynamics have profound implications for constitutional review. Legal doctrines such as proportionality analysis, as established in *Puttaswamy v. Union of India* [Supreme Court of India, 2017], presuppose that state intrusions can be measured, minimized, and remedied. However, as Bhatia [2019] argues, modern digital systems frequently violate the structural assumptions upon which these legal doctrines rest.

A court cannot apply proportionality review to a system whose execution is opaque (**CODE**). A legislature cannot remedy harm if the mechanism of harm is an irreversible digital transaction (**EXIT**). A regulator cannot oversee a system if they lack the technical capability to inspect it independent of the operator (**AUDIT**).

Where digital systems block these channels structurally, the rule of law becomes a legal fiction. External review mechanisms cannot function meaningfully if the internal architecture is designed to resist observation and modification. Therefore, **corrigibility** should be understood not merely as a technical specification, but as the *evidentiary precondition* required for the law to take hold of the machine.

5.4.1 The Rule of the Ledger

Digital execution introduces a rival governance paradigm: the **Rule of the Ledger**, in which authority is exerted through synchronized, self-executing artifacts rather than mediated institutional processes. Under this paradigm, the technical capacity for instantaneous state change (account freezes, entitlement adjustments) can be embedded in the execution layer; the institutional channels of review (courts, ombudsmen, bicameral procedural delays) operate on distinct and much slower time scales. The consequence is **Universal Failure Propagation**: when identity, payment, and entitlement layers synchronize, a single erroneous state transition in one layer can automatically and immediately propagate penalties across other layers.

Two structural implications follow. First, doctrines that presuppose reversible administrative acts (proportionality, injunctive relief) are ipso facto disabled if the necessary observability and reversal primitives are not present within the technical loop (**CODE**, **AUDIT**, **GOVERN**). Second, legal remedies become performative unless they are accompanied by verifiable, machine-enforceable restraints (signed chains of authority, multi-party escrow of execution privileges, or pre-authorized blocking hooks that operate within the same temporal envelope as the ledger itself).

The Rule of the Ledger therefore reframes the law-infrastructure relationship: courts and regulators cannot simply be invoked later; corrigibility requires embedding evidentiary and counter-majoritarian restraints into the system’s operational fabric. This is not a call for “autonomous legal machines” but for technical modes of constraint that render legal review materially possible rather than symbolic.

6 Corrigibility in Automated and Agentic Systems

This section serves as a **stress test** of the corrigibility framework. If the five tests are truly fundamental, they must apply not only to deterministic infrastructure but also to stochastic, learned systems where the assumptions underlying traditional verification collapse entirely.

The application of this framework to Artificial Intelligence and autonomous agents requires resolving a fundamental crisis of verification. The five tests, when applied to traditional software, assume a deterministic architecture where source code is the sole arbiter of behavior, inspection reveals logic, and exact reproduction is mechanically trivial. Learned systems (AI and ML models) dismantle these assumptions. In these systems, identical inputs may yield stochastic outputs; behavior is defined by training data rather than logic gates; and opacity is often an inherent property of the representation rather than a contrivance of the operator.

However, the collapse of deterministic transparency does not invalidate the structural necessity of corrigibility. To the contrary, as the system’s internal logic becomes less intelligible to human operators, the external capacity for correction becomes more critical. **The five tests remain invariant; only the evidentiary standard changes.**

This invariance under stress confirms that the framework captures structural properties of accountability, not implementation details of any particular technology.

6.1 The Invariance of the Standard

Proposition 1. *The five corrigibility tests remain invariant for learned systems. What changes is the verification method.*

The table below illustrates how the mechanical verification of each test adapts to the probabilistic nature of AI without softening the structural requirement.

Test	Deterministic Verification	Learned System Verification
EXIT	Can refuse the system	Disclosure + non-AI alternative
CODE	Source code determines behavior	Layered transparency across training pipeline
AUDIT	Inspect logic and outputs	Statistical bounds + continuous monitoring
GOVERN	Maintainer and RFC process	Governance over objectives and value tradeoffs
FORK	Copy code and deploy	Resource forkability (compute + data)

Table 4: Verification methods for deterministic vs. learned systems

6.2 What Is “Source” in a Learned System?

A central challenge in complying with the **CODE** test is redefining the object of inspection. In a learned system, behavior is distributed across a lineage of distinct artifacts. The inference code is merely the execution engine; the actual behavioral logic is encoded in the weights, which are in turn downstream of the training corpus, filtering decisions, and RLHF preferences.

Therefore, publishing inference code alone does not satisfy the **CODE** test. It allows an auditor to run the machine, but not to understand why it produces specific outputs. To bridge this gap, compliance requires structured disclosure across the training pipeline, such as **Data Sheets for Datasets** [Gebru et al., 2021] and **Model Cards** [Mitchell et al., 2019]. These artifacts function as the “source code” for structural evaluation; withholding them renders the system a black box.

6.3 The Temporal Variety Gap

A central theoretical contribution of this framework is the identification of the **Temporal Variety Gap**—a fundamental challenge that distinguishes learned systems from traditional infrastructure and explains why AI corrigibility requires ongoing governance, not merely deployment-time verification.

Unlike traditional software, which remains static until explicitly patched, learned systems possess a distinct vulnerability related to time. The variety of the controller (V_c) is frozen at the moment of training parameterization (θ), while the variety of the world it operates upon (V_d) continues to expand.

$$\Delta(t) = V(\text{disturbance}; t) - V(\text{controller}; \theta) \quad (14)$$

Without mechanisms for retraining or fine-tuning accessible to the community, the variety gap $\Delta(t)$ widens over time. Thus, AI corrigibility is not a deployment property, but a *persistence condition*.

6.4 Resource Barriers to Forkability

The right to **FORK** a learned system is constrained by physical resources rather than just copyright. In traditional software, the cost to fork is the cost of storage; in AI, the cost to fork is the cost of compute.

If a community has the legal right to fork a model but lacks access to the original training data or the massive compute clusters required to reproduce it, the “Fork” is illusory.

Claim 1. *Legal permission to fork without access to compute or data is meaningless. This is an infrastructure question, not a licensing question.*

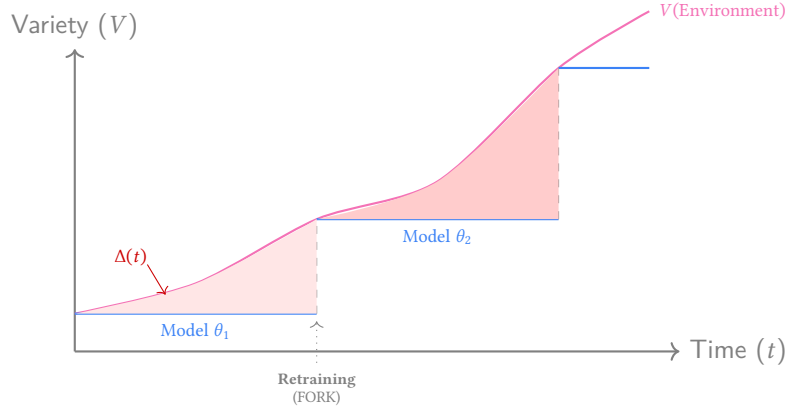


Figure 3: **The Temporal Variety Gap.** Environmental variety $V(e)$ evolves continuously (curve). A fixed model θ (stepped line) diverges from reality. The shaded area represents accumulated error. Only **FORK** rights allow the gap to be closed via retraining.

Layer	Cost	Barrier
Inference code	Minimal	None
Running open weights	Low	Minor
Fine-tuning	Moderate	Economic
Retraining from scratch	Very high	Structural

Table 5: Resource barriers to forking learned systems

6.4.1 The Compute Capture Theorem

The resource barrier to AI forkability can be formalized as a structural constraint:

Theorem 1 (Compute Capture). *Let C_{train} be the compute cost required to retrain a model, and $C_{accessible}$ be the compute accessible to plausible non-operator actors (academic clusters, government research consortiums, distributed compute pools). Training forkability is functionally impossible when:*

$$C_{train} > \kappa \cdot C_{accessible} \quad (15)$$

where κ is an accessibility multiplier (policy constant, typically $1 \leq \kappa \leq 2$).

This theorem formalizes what “open-weights AI” obscures: releasing weights while hoarding compute is **inference forkability without training forkability**. If OpenAI or Meta releases weights but C_{train} exceeds 100× the capacity of global academic compute, $FORK_{training} = 0$.

The formal determination becomes:

$$FORK_{training} = 1[C_{train} \leq \kappa \cdot C_{accessible}] \wedge 1[data_manifest_available] \quad (16)$$

where $1[\cdot]$ is the indicator function and `data_manifest_available` requires Data Sheets for Datasets [Geburu et al., 2021] with full provenance.

6.4.2 Compute as Governance Infrastructure

The Compute Capture Theorem implies that **compute is itself DPI**. Legal forkability is insufficient absent practical compute accessibility. This motivates several architectural responses:

1. **Public Compute Nodes:** State-sponsored clusters with governance boards (academia + civil society + government), prioritized quotas for public interest retrains and independent audits. These function as research infrastructure analogous to national laboratories.
2. **Mandatory Checkpoint & Cost Disclosure:** Require publication of FLOP estimates, cost-of-train, training checkpoints (or intermediates), and data manifests with privacy protections. These enable evaluation of C_{train} .

3. **Federated Training Pools:** Require DPI models affecting essential services to be trained via federated consortiums where multiple independent nodes share shards; no single node holds all data.
4. **Compute Vouchers:** Grants for accredited auditors and civic labs to attempt reproduction, establishing practical $C_{\text{accessible}}$.

The accessibility multiplier κ provides regulators a concrete policy lever: if the ratio $C_{\text{train}}/C_{\text{accessible}}$ exceeds the threshold, additional mitigation is required (e.g., mandated lighter-weight models, specialized models, or co-training access).

6.4.3 Minimum Evidence for Learned System Forkability

To prevent “FORK fails” from becoming a rhetorical claim rather than a verifiable determination, learned systems must satisfy specific evidentiary requirements:

Table 6: Evidence requirements for FORK in learned systems

Artifact	Status	Verification
Inference code	Required	Open source license; build reproducibility
Model weights	Required	Public download; hash verification
Training data manifest	Required	Data Sheets [Geburu et al., 2021] with provenance
Training checkpoint	Recommended	Enables fine-tuning without full retrain
Training code	Required	Includes preprocessing, hyperparameters
Compute cost estimate	Required	FLOP estimate or cloud cost documentation
Successful reproduction	Decisive	Third-party retrain within 10% of benchmark

Forkability Determination. A learned system passes FORK if:

1. All “Required” artifacts are publicly available under permissive terms
2. The compute cost estimate demonstrates feasibility for at least one non-operator entity (government research consortium, academic cluster, or distributed compute pool)
3. Either a successful third-party reproduction exists, OR the training code + data manifest enables reproduction in principle

Systems that publish weights but withhold training data or code achieve only **inference forkability**, not **training forkability**. Inference forkability permits running the model; training forkability permits correcting it. Only the latter satisfies FORK for governance purposes.

6.5 Agentic Scaling and Governance Denial of Service

As DPI transitions to support the “Agentic Economy,” corrigibility moves from a moral preference to an engineering requirement. Autonomous agents lack the biological resilience to handle bureaucracy.

We distinguish this from “AI Alignment.” Alignment asks: *Can the operator control the system?* Corrigibility asks: *Can the affected subject correct the system?* Structural legitimacy requires the latter.

6.5.1 Corrigibility as an API Contract for Agents

- **EXIT is Exception Handling:** To an agent, a mandatory system with no exit path appears as a logic deadlock.
- **CODE is Documentation:** Agents require inspectability of logic to form valid plans.
- **AUDIT is Observability:** An optimization agent cannot function without a reliable failure signal.
- **GOVERN is Constraint Discovery:** Advisory guidelines result in undefined behavior for software agents.

Claim 2. *Incorrigible infrastructure is structurally incompatible with AI automation.*

6.5.2 The Human Friction Buffer

Incorrigible infrastructures may appear stable under human load because humans absorb friction through delay, compliance, and informal workaround. Autonomous agents remove this buffering layer.

Illustrative Scenario. Consider an autonomous procurement agent that interacts with a mandatory payment API. On encountering a compliance flag, the agent receives an opaque error code. No machine-readable error taxonomy exists, and dispute resolution is human-mediated. The agent retries, triggering repeated failures and queue saturation. Human agents absorb such friction via manual override and phone calls; fleets of autonomous agents cannot. The result is a governance denial-of-service (GDoS) that cascades across the payment rail.

As agentic interaction scales, centralized governance mechanisms face **Governance Denial of Service (GDoS)**: the volume, speed, and strategic diversity of agent actions exceeds the system’s capacity to audit, interpret, and correct behavior.

This is not a security failure but a control-theoretic inevitability. Systems lacking distributed audit and rapid, binding governance lack the requisite variety to remain stable under agentic scaling. The correction velocity inequality (Section 5.2) predicts precisely this failure mode: when $\frac{dE}{dt} \gg \frac{dC}{dt}$, the system diverges. Agents accelerate $E(t)$; centralized governance cannot accelerate $C(t)$.

Claim 3. *Agentic scaling amplifies the correction velocity inequality. Systems that are marginally stable under human interaction rates become catastrophically unstable under agent interaction rates. The human friction buffer is not a feature; it is a hidden subsidy that masks architectural incorrigibility.*

7 Empirical Evaluation

To demonstrate the framework’s application, we evaluate systems across three categories: government infrastructure promoted as “DPI,” platform infrastructure claiming “openness,” and infrastructure that satisfies all five tests.

A Note on Utility. These evaluations assess structural accountability, not functional utility. A system can be simultaneously useful and incorrigible. Aadhaar may enable efficient service delivery; this does not establish that it permits correction by those it affects. The framework evaluates governance architecture, not operational performance. Utility and accountability are orthogonal dimensions: neither implies the other.

7.1 Government Infrastructure: The Trap of Partial Compliance

Government-operated systems currently designated as Digital Public Infrastructure frequently exhibit a dangerous failure mode: they achieve partial compliance on technical tests (EXIT, CODE, AUDIT) while failing structurally on governance tests (GOVERN, FORK). By capturing the monopoly on execution, they render the feedback loop inoperable.

Pattern: X-Road (4/5) fails only EXIT. UK Open Banking (2/5) passes AUDIT and GOVERN but fails CODE: the API *specification* is public, but the bank *implementations* are proprietary. Specification $\neq$ Code. You can see the interface; you cannot inspect the execution.

7.2 Platform Infrastructure: The Illusion of Openness

In the private sector, systems often leverage “open source” branding while retaining centralized control. This decoupling of artifact transparency from governance reality creates a distinct failure pattern: the artifacts are public; the decisions are not.

Pattern: Asymmetric Openness. These systems deploy strategic openness: releasing artifacts for ecosystem capture while retaining governance. This satisfies CODE (inspection possible) while failing GOVERN (correction blocked). The framework distinguishes artifact transparency (can you see the code?) from governance accountability (can you correct the system?). Open weights, open protocols, and open standards can each mask closed correction loops. “Open weights” $\neq$ “open.” “Open source” $\neq$ “accountable.” The artifact is open. The correction loop is closed. Seeing the machine does not govern it.

7.3 Corrigible Infrastructure

For contrast, Table 9 summarizes eighteen systems that pass all five tests. A striking pattern emerges: these systems are predominantly non-essential infrastructure. No individual’s survival depends on accessing Linux or Let’s Encrypt. This correlation between corrigibility and non-essentiality is not coincidental; it reflects the structural tension analyzed in Section 7.6.

Fork as Correction: OpenSearch, Valkey, LibreOffice, and MariaDB demonstrate Ashby’s Law in practice. Each emerged when governance variety collapsed: Oracle’s Sun acquisition (2010) triggered LibreOffice and MariaDB;

Table 7: **Evaluation of Government Infrastructure.** Centralized execution leads to consistent structural failure despite “open standards” rhetoric.

(a) Aadhaar (India) – Mandatory ID

Test	Result	Evidence
EXIT	×	Disproportionate penalty: exclusion from PDS, banking, SIM [Khera, 2019].
CODE	×	Execution logic opaque: proprietary matcher, closed algorithms.
AUDIT	×	External testing prohibited: independent error measurement illegal.
GOVERN	×	No binding constraint: UIDAI is both operator and regulator.
FORK	×	No legal/technical/economic means: total centralized capture.

(b) UPI (India) – Payment Rail

EXIT	PARTIAL	Reduced penalty: cash exists but network effects coerce.
CODE	PARTIAL	Partial inspection: protocol specs open, switching logic closed.
AUDIT	PARTIAL	Limited testing: regulator audits, public measurement blocked.
GOVERN	×	No binding constraint: banks own NPCI, regulate themselves.
FORK	×	No economic means: hub-and-spoke requires NPCI cooperation.

(c) Pix (Brazil) – Central Bank Payment

Test	Result	Evidence
EXIT	PARTIAL	Reduced penalty: cash and cards exist; digital-only services coerce adoption.
CODE	PARTIAL	Partial inspection: API specification public; central routing logic closed.
AUDIT	PARTIAL	Limited testing: aggregate statistics public; independent error measurement blocked.
GOVERN	×	No binding constraint: BCB sets and enforces Pix rules unilaterally.
FORK	×	No reproduction means: central switch requires BCB; no independent implementation.

(d) X-Road (Estonia/Nordic) – Data Exchange Layer

Test	Result	Evidence
EXIT	×	Disproportionate penalty: exclusion from government services.
CODE	✓	Execution logic inspectable: MIT source on GitHub.
AUDIT	✓	External testing possible: transaction logs, documented APIs.
GOVERN	✓	Binding constraint: NIIS treaty, MIT license irrevocable.
FORK	✓	Legal + technical + economic: 25+ country deployments prove feasibility.

(e) UK Open Banking – Regulated API Standard

EXIT	PARTIAL	Can revoke API consent; cannot exit banking system.
CODE	×	Execution logic not inspectable: bank implementations proprietary.
AUDIT	✓	External testing: FCA oversight, TPP certification.
GOVERN	✓	Binding constraint: CMA order, PSD2 regulation.
FORK	PARTIAL	Legal rights exist; economic means require regulatory recognition.

Table 8: **Evaluation of Private/Platform Infrastructure.** Technical openness (Source/Weights) does not guarantee structural accountability.

(a) Open-Weights AI (Llama, etc.)		
Test	Result	Evidence
EXIT	✓	No disproportionate penalty: can run locally, switch models freely.
CODE	PARTIAL	Partial inspection: weights public; training data and RLHF opaque.
AUDIT	PARTIAL	Limited testing: black-box probing possible; lineage unverifiable.
GOVERN	×	No binding constraint: firm sets safety/capability trade-offs unilaterally.
FORK	PARTIAL	Partial means: fine-tuning possible; retraining blocked by compute cost.
(b) Android (AOSP + GMS)		
Test	Result	Evidence
EXIT	PARTIAL	Moderate penalty: device portable; ecosystem lock-in creates friction.
CODE	PARTIAL	Partial inspection: AOSP open; Play Services and Identity closed.
AUDIT	PARTIAL	Limited testing: OS auditable; Google Services behavior opaque.
GOVERN	×	No binding constraint: Google sets roadmap and certification unilaterally.
FORK	PARTIAL	Partial means: LineageOS exists; app ecosystem requires GMS.
(c) Signal Protocol		
Test	Result	Evidence
EXIT	✓	No disproportionate penalty: can switch to alternatives freely.
CODE	✓	Full inspection: client and server source available (GPL/AGPL).
AUDIT	✓	External testing: cryptographic properties independently verifiable.
GOVERN	×	No binding constraint: Foundation board sets direction unilaterally.
FORK	×	No economic means: code forkable but no federation locks network effects.

unilateral license changes triggered OpenSearch (2021) and Valkey (2024). The ecosystem restored requisite variety through forking.

The framework is governance-agnostic: corporate participation is not disqualifying. Kubernetes (CNCF), Hyperledger, and Let’s Encrypt all have significant corporate involvement while passing all tests. The test is whether governance permits correction, not who participates.

License vs. Governance: Redis Ltd. added AGPLv3 in 2025, restoring open licensing. But license is orthogonal to governance. *License* determines what you may do with code; *governance* determines who decides what it becomes. Redis passes one; Valkey passes both.

7.4 The Taxonomy of Open-Washing

Systems that fail these structural tests frequently employ vocabulary to obscure their closure:

1. **Standard-Washing:** Adopting open technical standards (like ISO/W3C) as proxy for governance accountability.
2. **Open-Washing:** Releasing peripheral SDKs while keeping core execution logic proprietary.
3. **Multilateral-Washing:** Using international endorsements (G20, WB) to substitute for technical auditability.
4. **Safety-Washing:** Justifying opacity by claiming inspection poses a security risk.
5. **Consent-Washing:** Obtaining formal consent under conditions of economic coercion.
6. **DID-Washing:** Deploying decentralized identifiers without decentralized governance (see Section 7.4.1).

System	Steward	Score	Why Corrigible
Linux Kernel	Linux Foundation	5/5	Fully reproducible. Full triad (legal, technical, economic) exists. Independent distributions prove forkability.
Let's Encrypt	ISRG	5/5	Open source CA, IETF standards, nonprofit 501(c)(3)
Wikipedia	Wikimedia Foundation	5/5	CC BY-SA content, public edit history, community RfC governance
Matrix Protocol	Matrix.org Foundation	5/5	Apache 2.0, federated by design, self-hostable
Bluesky (AT Protocol)	Bluesky PBC	5/5	MIT/Apache, portable DID identity, self-hostable PDS
PostgreSQL	PGDG	5/5	BSD-like license, public mailing lists, major forks exist
IPFS	Protocol Labs	5/5	MIT/Apache, public IPIP process, no token governance
Bitcoin	Decentralized	5/5	MIT source, public BIP process, proven forks (BCH, BSV)
Kubernetes	CNCF	5/5	Apache 2.0, KEP process, distributions (OpenShift, k3s)
Firefox	Mozilla Foundation	5/5	MPL 2.0, nonprofit manifesto, forks (LibreWolf, Tor Browser)
Apache HTTP	Apache Foundation	5/5	Apache 2.0 license, ASF governance, powers 30% of web
Apache Kafka	Apache Foundation	5/5	Apache 2.0, KIP process public, Fortune 500 standard
OpenSearch	Linux Foundation	5/5	Forked from Elasticsearch; Apache 2.0, multi-vendor governance
Valkey	Linux Foundation	5/5	Forked from Redis; BSD license, community governance restored
Hyperledger	LF Decentralized Trust	5/5	Apache 2.0 framework; institutional deployments require separate evaluation
LibreOffice	Document Foundation	5/5	Forked from OpenOffice; LGPL/MPL, nonprofit governance
MariaDB	MariaDB Foundation	5/5	Forked from MySQL; GPL, foundation governance
Eclipse IDE	Eclipse Foundation	5/5	EPL 2.0, 386 member orgs, committer-led governance

Table 9: Infrastructure systems satisfying all five corrigibility tests. All systems demonstrate full EXIT, CODE, AUDIT, GOVERN, and FORK compliance.

7. **Climate-Washing:** Using environmental rhetoric to legitimize structural changes whose primary effects extend beyond genuine climate mitigation (see Section 7.4.2).

7.4.1 DID-Washing: The Layer Confusion Problem

Decentralized Identifiers (DIDs) present a distinct challenge to corrigibility evaluation because they separate technical decentralization from governance decentralization across multiple architectural layers. The W3C DID Core 1.0 specification [W3C, 2022] defines DIDs as “globally unique identifiers” that “do not require a centralized registration authority.” This technical claim is accurate. However, a system can implement DIDs at the identifier layer while retaining centralized control at every layer that determines actual utility.

Identity infrastructure comprises at least four functional layers, each with independent governance:

Layer	Function	DID-Washing Failure Mode
Identifier	Generate and resolve DIDs	Often decentralized, but meaningless alone
Credential	Issue verifiable claims about the subject	Usually centralized: single issuer dominates
Trust	Determine which credentials are accepted	Controlled by policy, trusted lists, or law
Revocation	Invalidate credentials or identifiers	Often requires issuer cooperation

Table 10: DID-washing failure modes by layer

A DID that resolves correctly but is not accepted by any relying party is functionally equivalent to no identity at all. The identifier is “self-sovereign”; the identity utility is not.

Empirical Examples. The EU Digital Identity Wallet (EUDI) under eIDAS 2.0 [European Union, 2024] illustrates sophisticated DID-washing. The architecture employs selective disclosure credentials and cryptographic proofs superficially similar to DID/VC ecosystems. However:

- Credential issuance for Person Identification Data is an exclusive government function.
- Trust determination requires registration in Member State Trusted Lists.
- Wallet certification depends on government-approved Qualified Trust Service Providers (QTSPs).
- Trusted Execution Environment dependency hands cryptographic key governance to mobile OS vendors (Google, Apple).

The result: users control their container (the wallet) but not their capability (the credentials). A journalist critical of their government holds a “decentralized” identifier that can be rendered useless by a single administrative act: revocation from the trusted list.

Consent-Washing: The DigiLocker Case. India’s DigiLocker exhibits a simpler failure mode. It does not claim to use DIDs; it uses centralized URIs and mandatory Aadhaar linkage. However, promotional language around “user consent” and “digital empowerment” obscures the structural reality: users cannot register without Aadhaar, cannot store documents outside government-controlled infrastructure, and cannot port their identity to alternative systems. This is not DID-washing but *consent-washing*: the vocabulary of user control applied to a system that offers none.

Implications for Framework Application. DID-washing exploits a gap in naive corrigibility evaluation. A system may appear to pass CODE (the DID spec is public) and FORK (anyone can implement the spec) while failing at the trust layer where actual power resides.

The framework must be applied layer by layer (see Section 4.8). For identity systems:

- CODE must be evaluated at each layer: identifier resolution, credential issuance logic, trust framework rules, and revocation mechanisms.
- GOVERN must ask: who controls the acceptance policy? If a single authority determines which credentials are valid, governance is centralized regardless of identifier architecture.
- FORK must ask: can a community reproduce the trust framework, not merely the identifier scheme? If relying parties are legally required to accept only government-issued credentials, forkability is illusory.

A system passes the DID-washing test only if decentralization extends through the full trust stack: issuance, verification, acceptance policy, and revocation governance. Technical decentralization at the identifier layer with governance centralization at the trust layer is structural theater.

Claim 4. *Identifier decentralization is not system decentralization. The corrigibility of an identity system is determined by its weakest layer across any test dimension.*

7.4.2 Climate-Washing: Environmental Rhetoric as Deflection

Definition 6 (Climate Washing). *The use of climate rhetoric or urgency to legitimize actions, policies, or structural changes whose primary effects extend beyond genuine climate mitigation, whether for reputational, financial, institutional, or governance advantage.*

A variant of safety-washing has emerged in DPI discourse: claims that digital infrastructure is inherently “sustainable” or “green” because it replaces paper, reduces travel, or enables “efficient” resource allocation. Climate washing deploys environmental rhetoric to deflect accountability critiques.

The structural analysis reveals three problems:

1. Hidden Energy Footprint. Digital infrastructure has non-trivial energy and material costs: data centers, network equipment, device manufacturing, and e-waste. The claim that “digital is green” compares against a paper baseline while ignoring these embodied costs. A proper lifecycle analysis would compare total system footprints, including the rebound effects of increased transaction volume enabled by digital convenience.

2. Transparency Failure. Climate claims for DPI systems typically fail the CODE test: the energy consumption data, carbon accounting methodology, and lifecycle assumptions are not publicly auditable. When NPCI claims UPI reduces carbon emissions, the underlying calculations are not verifiable. This is precisely the audit-theatre pattern identified in Section 5.3.2: a claim of virtue without the evidentiary infrastructure to support it.

3. Deflection Function. Climate rhetoric serves a political function: it reframes accountability critiques as opposition to environmental progress. The implicit argument is: “If you criticize Aadhaar’s surveillance, you are against paperless government and therefore against the environment.” This framing collapses distinct concerns (privacy, accountability, environmental impact) into a single axis and positions critics as anti-progress.

The framework’s response is structural: environmental claims, like safety claims, must satisfy the five tests. A system cannot claim environmental legitimacy if its energy footprint is not auditable (CODE, AUDIT), if affected communities cannot contest the methodology (GOVERN), or if lock-in prevents migration to more efficient alternatives (EXIT, FORK). “Green DPI” that fails these tests is not green infrastructure; it is climate washing.

	EXIT	CODE	AUDIT	GOV	FORK	Result
Aadhaar	×	×	×	×	×	0/5
UPI	~	~	~	×	×	0/5
Pix	~	~	~	×	×	0/5
Llama etc.	✓	~	~	×	~	1/5
Linux	✓	✓	✓	✓	✓	5/5
Let's Enc.	✓	✓	✓	✓	✓	5/5
K8s	✓	✓	✓	✓	✓	5/5
Bitcoin	✓	✓	✓	✓	✓	5/5
+15 more	See Table 9					
✓ = Pass ~ = Partial × = Fail						

Figure 4: Comparative evaluation summary. Systems labeled “DPI” (above line) fail structural tests. Corrigible infrastructure (below line) passes all five. See Table 9 for complete list.

7.5 Implications

The contrast is instructive. Aadhaar and UPI, promoted as model DPI, fail all tests. The eighteen systems in Table 9, rarely described as “DPI,” pass all tests.

This suggests the “public” in Digital Public Infrastructure often refers to *population-scale deployment* rather than *public accountability*. The framework distinguishes these: scale is not governance.

All systems that pass share common properties:

- Open source with permissive or copyleft licensing
- Nonprofit or community governance structures
- Protocol standardization through open processes (IETF, LKML)
- No monopoly on essential services

Scale does not require capture. Universality does not require opacity. The choice is architectural.

7.5.1 The DPI Trilemma

The empirical pattern suggests a structural constraint we call the **DPI Trilemma**. A state-operated DPI can strongly realize any two of the following three properties, but not all three simultaneously:

1. **Sovereignty**: local control over standards, execution, and data residency;
2. **Scale**: population-level ubiquity and mandatory integration across essential services;
3. **Neutrality**: non-discriminatory access and competitive fairness at the execution layer.

Formally: under middle-power constraints (limited leverage over global platform incumbents), achieving both Sovereignty and Scale requires centralized enforcement of standards and routing. That centralization tends to compress the governance options available at the execution layer and thus erodes Neutrality. Conversely, systems that safeguard Neutrality (Linux, Bitcoin) typically lack mandatory scale or are provisioned in ways that preserve distributed governance.

India’s experience is illustrative: the design choices that delivered an “Atmanirbhar” stack and near-universal reach (Sovereignty + Scale) produced a structural dependency on a small set of switching and credentialing authorities, which in turn materially reduced neutrality in execution (see Aadhaar/UPI evaluations, Table 7). The Trilemma reframes these outcomes: the observed failures are not merely implementation mistakes but predictable architectural trade-offs.

Any prescriptive response must therefore treat neutrality as an architectural variable (portability, multi-issuer requirement, anti-exclusive procurement), not an aspirational outcome. The Functional Exit Equivalence remedies in Section 7.6.3 address precisely this constraint.

7.6 The Essential Services Problem

A pattern emerges from the preceding analysis: systems that pass all five tests are disproportionately non-essential infrastructure. Linux, PostgreSQL, Kubernetes, and Let’s Encrypt power critical operations, but no individual’s survival depends on accessing them. The systems promoted as DPI, which mediate access to food, banking, healthcare, and mobility, consistently fail.

This correlation reflects a structural tension between essentiality and corrigibility.

Proposition 2 (Essentiality-Corrigibility Tension). *For any infrastructure system S where participation is a prerequisite to survival, the EXIT test cannot be satisfied.*

Proof. EXIT requires that refusal incur no disproportionate penalty (Section 4, Test 1). Let S be essential: participation is required for access to food, shelter, healthcare, or legal identity. Refusal of S therefore entails exclusion from survival necessities. Exclusion from survival necessities is a disproportionate penalty by any reasonable standard. Therefore, EXIT fails. □

This proposition formalizes the empirical observation: among the systems evaluated, those that pass all five tests are precisely those where participation remains genuinely voluntary.

Corollary 1 (No Exemptions from Essentiality Tension). *The following do NOT constitute valid exemptions from Proposition 2:*

1. **Emergency necessity:** *Pandemics, wars, and crises do not suspend the requirement for correction channels. Systems deployed under emergency conditions that block EXIT remain incorrigible; the emergency explains but does not justify the structural failure.*
2. **Efficiency gains:** *That a mandatory system delivers services faster or cheaper than alternatives does not restore EXIT. Efficiency is orthogonal to corrigibility.*
3. **Democratic authorization:** *Legislative mandates do not convert incorrigible systems into corrigible ones. A democratically enacted prison is still a prison.*
4. **Benevolent intent:** *Good-faith operation does not substitute for structural accountability. The framework evaluates architecture, not intentions.*

7.6.1 The Essentiality Trap

Essential services create the economic coercion that undermines EXIT. When refusal means exclusion from survival, the feedback loop breaks regardless of other properties. Hirschman [1970] observed that exit is effective precisely because it is voluntary. The credible threat of departure disciplines the system. When departure means death (literal or social), exit ceases to function as feedback. It becomes self-exclusion.

The systems in Table 9 avoid this trap because alternatives exist. A developer who dislikes the Linux kernel’s direction can migrate to BSD without losing the ability to compute. A journalist who distrusts Let’s Encrypt can obtain certificates elsewhere. The existence of functional alternatives preserves the error signal.

Aadhaar has been made prerequisite to existence. Documented cases demonstrate the consequence: at least 27 starvation deaths in India between 2015 and 2018 were attributed to Aadhaar-related exclusion from food rations [Khera, 2017]. These deaths are not anomalies; they are the structural prediction of a system that blocks EXIT while governing essential services.

7.6.2 Functional Exit Equivalence (FEE)

Proposition 2 establishes that literal exit is impossible for essential services. However, the **purpose** of EXIT is not departure itself; it is the generation of a high-bandwidth, credible error signal that disciplines the system. This section introduces **Functional Exit Equivalence (FEE)** as the architectural substitute for literal exit when a service is essential.

Definition. An essential system S satisfies FEE if compensatory architectural guarantees recreate the same effective error-signal strength that exit provides in non-essential systems.

Let:

- $E_{\text{exit}}^{\text{baseline}}$ = effective error-signal strength produced by real exit (non-essential case)
- $E_{\text{alt}}(S)$ = effective error-signal strength produced by compensatory mechanisms (essential case)

$$\text{FEE}(S) \iff E_{\text{alt}}(S) \geq \theta_E \cdot E_{\text{exit}}^{\text{baseline}} \quad (17)$$

where θ_E is a policy calibration constant (recommended range: 0.8–1.0).

Operationalizing E . The error-signal strength E can be operationalized as a weighted sum:

$$E = \alpha_1 \cdot \text{effective_loss} + \alpha_2 \cdot \text{probability_of_departure} + \alpha_3 \cdot \text{velocity_of_response} + \alpha_4 \cdot \text{legal_remedy_speed} \quad (18)$$

where the weights α_i are calibrated via pilot audits. All variables are operationalized such that higher values indicate greater corrective pressure on the operator: a higher velocity of legal response, for instance, increases E , thereby satisfying the FEE condition. This formulation connects back to Ashby’s Law: FEE is an engineered mechanism to maintain requisite variety in the controller when the natural market variety (EXIT) is suppressed.

7.6.3 Design Responses to Proposition 2

The tension admits architectural resolutions that achieve FEE:

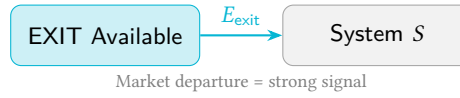
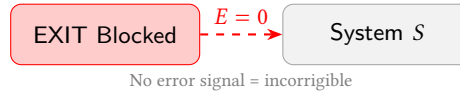
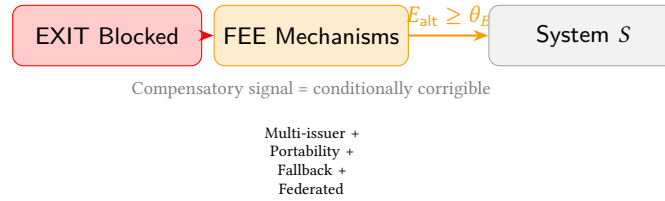
Non-Essential System:**Essential System (No FEE):****Essential System (with FEE):**

Figure 5: **Functional Exit Equivalence (FEE)**. Top: Non-essential systems generate error signals through market exit. Middle: Essential systems without FEE block error signals entirely. Bottom: Essential systems with FEE use compensatory mechanisms to restore error signal strength.

1. Protected Alternatives (Statutory Fallback). Essential services can satisfy FEE if law guarantees functional non-digital equivalents. A model statutory clause:

“No person shall be required as a condition of receiving any essential public benefit to be exclusively dependent on a single digital system; the State shall maintain a legally enforceable analogue alternative and an interoperable credential acceptance guarantee.”

Cash preserves UPI’s partial EXIT; eliminating cash would collapse UPI to Aadhaar’s structural status. Corrigibility for essential services requires that non-digital pathways remain legally protected, not merely tolerated.

2. Multi-Issuer Requirement. Procurement and regulation shall require at least two legally independent credential issuers for each region or service class. No single issuer may be sole gatekeeper for eligibility. This creates credible switching alternatives that generate E_{alt} .

3. Credential Acceptance Diversity. Public services must accept at least three credential classes (state, municipal, NGO/financial) to satisfy eligibility requirements. This prevents single-point-of-failure capture.

4. Binding Multi-Stakeholder Charter. A legal instrument that locks governance rules, with amendments requiring supermajority approval, judicial review pathway, and defined emergency suspension conditions.

5. Funded Fallback Operations. Government must fund and staff analogue fallback channels wherever the digital system is primary, not as legacy support but as structural accountability infrastructure.

Federated Implementation. Rather than a single provider of essential infrastructure, services could be delivered through multiple independent implementations of open protocols. This is analogous to email (SMTP) rather than messaging (proprietary platforms).

Under this model, no single implementation would be essential; the protocol would be. A citizen excluded from one provider could obtain equivalent service from another. Estonian X-Road approaches this architecture: the protocol is open, implementations are distributed across institutions, and the Nordic Institute for Interoperability Solutions (NIIS) provides multi-stakeholder governance.

X-Road demonstrates that scale and corrigibility are compatible. It passes CODE (MIT-licensed), AUDIT (transaction logs, public enforcement against misuse), GOVERN (multi-national NIIS), and FORK (deployed in 25+ countries). It fails full EXIT but achieves FEE through strong performance on the other four tests plus federated implementation.

7.6.4 The Finding

The absence of fully corrigible essential services is not a limitation of this framework; it is the framework's central finding. The question "Can essential DPI be fully corrigible?" is answered: **not under architectures that eliminate alternatives, but achievable through FEE-compliant designs**. Proposition 2 is not a dead end; it is a design specification.

8 Discussion

This section addresses three themes: methodological foundations, political economy, and implementation pathways.

8.1 Methodological Foundations

8.1.1 The Cybernetics of Binary Evaluation

Critics often argue that this framework's binary pass/fail topology is too rigid for the complex reality of governance, proposing instead "maturity models" or graded scorecards. From a cybernetic perspective, we reject this gradation. Corrigibility is not a variable virtue; it is a **structural connectivity state**.

In a control system, a feedback loop is either closed or it is open. There is no such thing as a "mostly closed" loop. A wire that is disconnected by 1 millimeter transmits zero signal.

- A system that allows **AUDIT** (Sensing) but blocks **GOVERN** (Actuation) is an **Open Loop**. It observes errors but cannot fix them.
- A system that allows **GOVERN** but blocks **EXIT** is a **Coercive Loop**. It traps energy inside the system until it fails catastrophically.

Partial compliance creates "zombie infrastructure": systems that maintain the aesthetic of responsiveness while severing the actual capacity for stability. Therefore, a "Partial" score in this framework represents a **Fatal Structural Failure**. A system is either corrigible, or it is not.

The binary model is a deliberate analytical choice. Real systems exhibit continuous variation in feedback quality: channels may be partially blocked, signals degraded but not absent, corrections slow but not entirely unavailable. The binary classification discards this nuance for structural clarity.

This trade-off is justified on two grounds. First, the regulatory purpose: a system either meets the threshold for public legitimacy or it does not. Graduated compliance creates negotiation space that operators exploit. Second, the cybernetic purpose: a feedback loop either closes or it does not. Partial closure is a category error; a wire that transmits 50% of the signal does not produce 50% of the regulation.

For diagnostic purposes, gradation remains useful. Tables 11 and 12 specify continuous measurements such as `penalty_ratio`, `visibility`, and `fix_latency_observed`. The binary determination is the output; the continuous measurements are the inputs. Evaluators can examine the inputs to prioritize interventions even when the output is uniformly "fail."

8.1.2 Legitimacy Threshold vs Physical Claim

A critical clarification: the binary model is a **legitimacy threshold**, not a physical claim about feedback dynamics. Control theorists correctly observe that real systems exhibit continuous variation in loop gain, observability, and actuation authority. A sensor with 50% accuracy transmits *some* signal; partial actuation produces *some* correction.

The framework does not dispute this physical reality. It makes a normative claim: **below a threshold of feedback quality, a system forfeits the right to claim public legitimacy**. The threshold is binary because legitimacy is binary. A prison that permits appeals 10% of the time is still a prison.

Sensitivity Analysis. Let continuous inputs be $\{p, v, \tau, b, r\}$ (penalty ratio, visibility, latency, bindingness, resource ratio) as defined in Table 11. The binary determination function D is:

$$D = 1[p \leq p^*] \wedge 1[v \geq v^*] \wedge 1[\tau \leq \tau^*] \wedge 1[b = \text{true}] \wedge 1[r \leq r^*] \quad (19)$$

where $1[\cdot]$ is the indicator function and $\{p^*, v^*, \tau^*, r^*\}$ are threshold values. This formulation makes explicit that:

1. Continuous inputs are preserved for diagnosis
2. The AND-conjunction enforces the “weakest link” principle
3. Threshold calibration is a policy choice, separable from the structural argument

Systems near thresholds warrant scrutiny; systems far below thresholds across multiple dimensions are unambiguously incorrigible. The binary output serves regulatory clarity; the continuous inputs serve diagnostic priority.

8.1.3 Continuous Indicators for Diagnostic Use

The continuous measurements underlying binary determinations enable both threshold calibration and diagnostic prioritization. Table 11 specifies operational proxies; Table 12 formalizes the three-indicator protocol.

Table 11: Operational proxies for the five tests. Thresholds are illustrative and subject to calibration.

Test	Proxy Metric	Illustrative Threshold
EXIT	Penalty ratio p	$p \leq 1.2$
CODE	Visibility fraction v	$v \geq 0.9$
AUDIT	Mean time to detect (MTTD)	$\text{MTTD} \leq 24 \text{ hours}$
GOVERN	Bindingness + throughput τ	$\text{binding} = \text{true}, \tau \geq 1 \text{ per } 10\text{k users/day}$
FORK	Resource feasibility ratio r	$r \leq 10^{-3}$

Interpretation: Penalty ratio $p = \text{cost}(\text{non-digital alternative}) / \text{cost}(\text{digital})$. Visibility $v = \text{public LOC} / \text{total LOC}$ on critical execution path. Resource ratio $r = \text{estimated FLOP to retrain} / \text{aggregate community compute capacity}$.

Table 12: Three-indicator protocol: continuous score, binary flag, and threshold metric for each test.

Test	Continuous Score	Binary Flag	Threshold Metric
EXIT	penalty_ratio: cost(exit) / cost(stay)	legal_barrier: is exit criminalized?	Pass if $p \leq 1.2$ AND $\text{legal_barrier} = \text{false}$
CODE	visibility: public LOC / total critical path LOC	build_reproducible: deterministic build exists?	Pass if $v \geq 0.95$ AND $\text{build} = \text{true}$
AUDIT	observability: % transactions independently verifiable	access_permitted: third parties can test?	Pass if $o \geq 0.9$ AND $\text{access} = \text{true}$
GOVERN	override_resistance: historical rate of operator overrules	binding_charter: irrevocable legal instrument?	Pass if $r = 0$ AND $\text{charter} = \text{true}$
FORK	cost_ratio: fork cost / community resources	legal_rights: permissive license?	Pass if $c \leq 0.001$ AND $\text{rights} = \text{true}$

Inter-Rater Calibration. Before deployment, independent auditors should assess 3–5 systems using this protocol and compare determinations. Disagreements identify threshold calibration issues. Target: $\kappa \geq 0.8$ (Cohen’s kappa) for binary determinations.

8.2 Political Economy of Accountability

8.2.1 The Protocol Model of State

A common practical objection holds that governments cannot adopt corrigible infrastructure because they must retain control over sovereign functions. This objection relies on a category error: it conflates the *Platform Model* with the *Protocol Model*.

In the **Platform Model** (currently dominant), the state contracts a vendor to build a monolithic silo (e.g., existing digital ID systems). The vendor controls execution, the state attempts to control policy, and citizens are reduced to

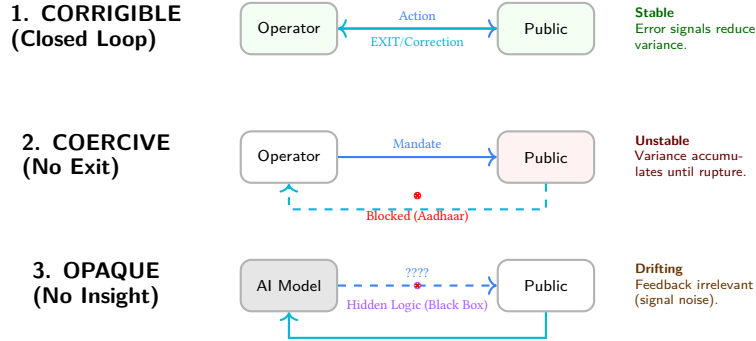


Figure 6: **Topologies of Failure.** (1) A corrigible system closes the loop, converting error signals into stability. (2) A coercive system blocks EXIT, trapping energy. (3) An opaque system obscures the action channel, making feedback unintelligible.

rows in a database. As Cohen [2019] argues, this model inevitably subverts the Rule of Law by replacing legal due process with optimized platform logic. In the **Protocol Model** (the corrigible alternative), the state adopts open, corrigible protocols and acts as a high-authority participant within them. The state issues credentials and verifies claims, but it does not enclose the infrastructure itself.

This distinction mirrors the architecture of the web. Governments do not need to build their own proprietary browsers to deliver services; they build sites on HTTP. Similarly, corrigible DPI implies that the state becomes an issuer of trust rather than a landlord of infrastructure. This model does not reduce state capacity; it enhances trust by making the state's operations verifiable.

8.2.2 Distributed Assessment Capacity

A corollary objection concerns resources: "Who audits?" The framework does not require every citizen to audit source code. It requires that the infrastructure allows *anyone* to audit, thereby enabling civil society, journalists, and specialized firms to perform this function on behalf of the public. Assessment capacity is itself infrastructure. When systems are designed to be inscrutable, this immune system of society withers.

8.3 Corrigibility in Extremis

Incorrigibility is often justified by the rhetoric of necessity, such as national security, anti-corruption, or emergency efficiency. This framework explicitly denies such exemptions. Corrigibility does not imply that substantive rules are negotiable; a system may enforce strict, non-negotiable limits (e.g., pandemic travel restrictions or environmental caps). However, the *structural capacity* to detect error, verify correct enforcement, and reverse wrongful harm must remain intact even in high-constraint environments. As safety engineering demonstrates, accidents in complex systems are caused precisely by the lack of appropriate constraints on component interactions [Leveson, 2011].

Claim 5. *No exemption from the five tests is granted on the basis of emergency conditions, planetary constraints, or decentralized architecture.*

This applies equally to decentralized systems. Cryptographic immutability is not a substitute for governance. A blockchain that permanently records a theft without a mechanism for consensus-based reversion is not "trustless"; it is strictly incorrigible. If a system allows for irrevocable harm without recourse, its decentralization is merely a distribution of unaccountability.

8.3.1 Political vs Structural Corrigibility

A critical objection holds that the framework applies market logic (exit/fork) to non-market systems. You cannot fork India.

The response requires distinguishing two modes of correction:

- **Political corrigibility** (voice via elections): how *states* are corrected
- **Structural corrigibility** (exit/fork): how *infrastructure* is corrected

States are corrected politically. DPI sits *below* politics; that is the problem. When DPI is captured by executive authority, political correction mechanisms do not reach it.

Aadhaar was not passed by Parliament until 2016 (as money bill to avoid Rajya Sabha). UIDAI operated for 7 years on executive authority alone. This is the gap the framework identifies: infrastructure that escapes democratic correction.

Democracies have *constitutional* correction mechanisms (elections, courts, amendments). DPI often lacks these. The Constitution can be amended; Aadhaar's architecture cannot be voted on.

8.3.2 The Distributional Question

The debate is not "scale vs sovereignty." The debate is **whose suffering counts**.

- Platform speed benefits the median user.
- Incorrigibility harms the marginal user.
- The marginal user has no voice because EXIT is blocked.

Corrigibility is a **distributional question**. The framework makes visible who bears the cost of system failures. When we ask "Is this system corrigible?" we are really asking: "When this system fails someone, can they do anything about it?"

The answer for Aadhaar is no. Documented starvation deaths in Jharkhand were not caused by "too much democracy"; they were caused by a system where the marginal user had no recourse when biometric authentication failed.

8.4 Pathways and Limits

8.4.1 Transition Paths

The framework diagnoses incorrigibility but does not prescribe political remedies. However, the structure of the five tests implies an ordering for transition: certain tests are prerequisites for others, and certain interventions have higher leverage.

8.4.2 Ordered Priorities

The tests form a dependency chain when considered as a transition sequence:

1. **EXIT first.** Restore exit before other tests. Without exit, no other reform creates genuine feedback. This requires either eliminating mandatory linkage laws or guaranteeing functional non-digital alternatives. EXIT is the error signal; without it, the system cannot sense its own failures.
2. **CODE before AUDIT.** Transparency must precede verification. Independent testing is meaningful only if the testing target is observable. Publishing audit results for a black box proves nothing about the box.
3. **AUDIT before GOVERN.** Verified error data creates political pressure for governance reform. Governance without measurement is arbitrary. There is no basis for determining whether constraints are adequate. The sensor must function before the actuator can be calibrated.
4. **GOVERN enables FORK.** Structural governance can mandate open licensing, data portability, and interoperability requirements that enable eventual forkability. GOVERN without FORK leaves the system captive to its current steward; FORK without GOVERN may produce fragmentation without accountability.

This sequence reflects the causal structure of the feedback loop: signal, sensing, transmission, actuation, selection. Interventions at later stages are ineffective if earlier stages are broken.

8.4.3 Phase Transitions

The transition is not gradual improvement. It is a series of phase transitions. Each test either passes or fails; the system remains incorrigible until all five pass. A system that achieves CODE but not EXIT has become transparent without becoming accountable. Users can observe the system's logic but cannot modify their participation.

This explains why open-washing is effective: partial progress creates the appearance of reform while preserving structural capture. The framework rejects partial credit precisely because partial feedback loops do not partially function.

8.4.4 Political Economy

The political economy of transition varies by test:

Table 13: Political economy of transition by test

Test	Transition Barrier	Concentrated Opposition
EXIT	Mandatory linkage laws	Operators lose captive users
CODE	Intellectual property claims	Vendors lose competitive moats
AUDIT	Security-through-obscurity claims	Operators lose narrative control
GOVERN	Sovereignty objections	Executives lose discretion
FORK	Typically least opposed	Requires only licensing and documentation

FORK is typically the least opposed transition because it requires only legal permission and technical documentation. It does not immediately threaten operator control. However, FORK without the prior four tests is meaningless: the right to reproduce a system you cannot inspect, verify, govern, or exit provides no actual power.

8.4.5 Cost Considerations

Transition is not costless. The European Commission [European Commission, 2021] estimated that public sector open-source adoption yields cost-benefit ratios above 1:4, but this ratio assumes successful implementation. Transition costs include personnel retraining, system compatibility engineering, procurement policy updates, and organizational change.

However, these costs must be weighed against the costs of continued incorrigibility:

- Litigation costs from unresolvable grievances
- Political instability from suppressed correction demand (Section 5.2)
- Technical debt from opacity-enabled shortcuts
- Vendor lock-in premiums from non-forkable dependencies
- Human costs: documented deaths, exclusions, and deprivations

The systems in Table 9 demonstrate that corrigibility at scale is economically sustainable. Linux, Kubernetes, and Let's Encrypt operate at population scale with resource models that have proven durable over decades.

8.4.6 Coordination Goods and Inherent Unforkability

The forking paradox presents a genuine limitation: if you can fork the Public Infrastructure, it may no longer be public.

A “public” good (like law or money) derives its value from universality. We all agree on the same truth. If the Identity System is “forkable,” and I fork it to create “Blue-Identity” while you use “Red-Identity,” we have destroyed the common knowledge required for society to function.

Concession: Identity, money, and law may be **inherently unforkable** coordination goods. The paper does not demand *actual forking*; it demands *credible threat of forking*. But for coordination goods, even the threat may undermine function.

This is a genuine tension the framework cannot fully resolve. The stable equilibrium may be: latent forkability + strong coordination incentives. Email works not because everyone forks SMTP, but because anyone *could*. Whether this equilibrium is achievable for identity infrastructure remains an open question.

8.4.7 Engaging “Regulate the Loop”

A sophisticated objection accepts the diagnosis but rejects the prescription:

“We accept your Diagnosis (The Loop is Broken). We reject your Prescription (Break the Platform). We choose to **Regulate the Loop** rather than Fork the Code, because the cost of Forking is Poverty.”

This argument has force. UPI moved from 0 to billions of transactions in 7 years. Email standardization took decades. India does not have decades. Poverty is now.

Where this lands:

- RBI *could* mandate error rate disclosure, demographic audits, grievance SLOs
- Political correction (parliament, courts) exists even without FORK
- Speed-to-scale tradeoff is real

Where this misses:

- “Regulate the loop” assumes trustworthy regulators. RBI reports to Finance Ministry. UIDAI is executive authority with no parliamentary oversight. Who regulates the regulator?
- In captured states, “regulate later” means “never”
- Gmail ≠ UPI: Gmail *earned* users with features: you CAN leave for Proton, Fastmail, self-hosted. UPI has *regulatory monopoly*: you CANNOT build a competing instant payment switch. Gmail is incorrigible by choice. UPI is incorrigible by law.
- “Regulate later” rarely happens: Aadhaar 2009 pilot → 2016 mandatory → 2018 Supreme Court → still no real audit. 15 years. Where’s the regulation?

8.5 Utility Does Not Equal Accountability

A system can be **useful AND structurally captured**. Both are true simultaneously.

UPI *has* democratized payments. The framework risks dismissing real-world impact if this is not acknowledged. But the question the framework asks is different: **When UPI fails someone, can they correct it?**

The answer is no. This is the point.

The framework measures **accountability structure**, not **utility**. A system can score 0/5 and still be beneficial. The score indicates *structural risk*, not *current harm*. A house built without fire exits can function perfectly for decades, until there is a fire.

9 Conclusion

Digital Public Infrastructure is not a specific technology; it is a claim of political legitimacy. It asserts that a system is sufficiently foundational, accessible, and accountable to serve as the substrate of civic life.

This paper argues that such legitimacy cannot be derived from self-declarations, “open standards” labeling, or multilateral endorsements. It relies on a specific cybernetic property: **Corrigibility**. The capacity of the governed to observe the rules, verify their execution, limit their reach, and, if necessary, reproduce the infrastructure to escape its capture.

This paper has proposed five rigorous tests to verify this capacity. The analysis demonstrates that systems currently touted as DPI models (Aadhaar, UPI, and proprietary AI) fail these tests, revealing them to be administrative enclosures rather than public goods. Conversely, the systems which actually run the world (Linux, Kubernetes, the Web) pass them structurally.

The choice is not between chaos and control. It is between **Open Loop** systems that drift toward fragility and authoritarianism, and **Closed Loop** systems that stabilize themselves through the correction of those they serve.

Systems that cannot be corrected by those they affect will eventually be corrected by forces they cannot control.

9.1 Limitations

This framework has the following limitations:

1. **Binary classification loses diagnostic information.** A system failing one test is treated identically to one failing all five for the purpose of the corrigibility determination. This is structurally correct (a broken feedback loop is broken regardless of where it breaks) but diagnostically impoverished. The schema (Appendix C) preserves gradation in fields like `penalty_ratio` and `visibility`, but the final determination collapses to binary.

2. **Governance evaluation is judgment-dependent.** CODE and FORK have clear verification criteria for code artifacts. GOVERN is harder to verify: governance claims require historical evidence of enforcement, and such evidence may be contested or incomplete. The evidentiary standards in Appendix D address this, but governance evaluation remains more interpretation-dependent than technical evaluation.
3. **No fully corrigible essential services are identified, but corrigible alternatives exist.** The systems in Table 9 are non-essential infrastructure. Estonian X-Road approaches corrigibility but fails full EXIT. However, population-scale corrigible infrastructure is not impossible: it is deployed. Email (SMTP) serves billions through federation. Matrix protocol powers government communications in France (Tchap) and Germany (BwMessenger). Bluesky’s AT Protocol demonstrates federated social infrastructure at scale with portable identity. Mastodon proves federated social media can grow. Bitcoin demonstrates that even payment infrastructure can operate at population scale without central authority, with proven forks (BCH, BSV) demonstrating FORK in practice. These systems achieve population scale while passing all five tests. The absence of corrigible essential services like identity and payment reflects political choices at system inception, not technical necessity. The research question is not “Can corrigible DPI exist at scale?” but “Why do governments choose incorrigible architectures when corrigible alternatives are proven?”
4. **Resource forkability for AI remains unresolved.** Section 6.4 identifies that legal rights without compute access may be permanently unfulfillable for large AI systems. The framework diagnoses this as a structural barrier to FORK but does not resolve the underlying resource asymmetry. Whether “compute as infrastructure” could make AI forkability meaningful is an open question.
5. **The timescale argument applies to external remedies.** Section 5.1 argues that bureaucratic correction is structurally too slow for digital-time errors. However, some corrigible systems (Bitcoin’s BIP process, Linux’s kernel mailing list) operate on human timescales and are accepted as valid governance. The distinction is that these mechanisms are internal to the technical community, not external bureaucracies. The timescale critique applies to external remedies (courts, ombudsmen), not to all deliberative processes.
6. **Network effects are acknowledged but not formally modeled.** The FORK test requires that reproduction be economically feasible, but network effects can make formally forkable systems practically unforkable. Signal’s code is fully open; its network is not federated. A Signal fork without users is not a meaningful check on Signal Foundation governance. The framework flags this but does not provide a formal treatment of network-effect capture.

9.2 Future Work

Several extensions warrant investigation:

1. **Empirical measurement of correction velocity.** The velocity inequality (Section 5.1) predicts system instability when correction latency exceeds error accumulation rate. Historical case studies (PDS exclusion incidents, payment system outages, identity fraud waves) could quantify the threshold at which systems become ungovernable.
2. **International comparative analysis.** This paper focuses substantially on India Stack. Systematic comparison with Estonian X-Road, Singapore’s National Digital Identity, the EU Digital Identity Wallet, and Brazil’s Pix would test the framework’s generality across political systems.
3. **Political economy of architecture choice.** Corrigible population-scale infrastructure exists (SMTP, Matrix, Bitcoin). Governments have deployed federated FOSS for sensitive communications (France, Germany). Yet most DPI initiatives choose centralized, incorrigible architectures. What political economy factors drive this choice? Hypotheses include: state preference for legibility (Scott), vendor capture, path dependence from platform-era defaults, and the conflation of control with security. Comparative case studies of architecture decisions at system inception would illuminate why corrigibility is rejected when alternatives are proven.
4. **Economic modeling of transition costs.** What does it cost to make an incorrigible system corrigible? Infrastructure-specific cost modeling, particularly for systems with existing vendor lock-in, would inform policy prioritization.
5. **AI-specific verification protocols.** The CODE test for learned systems (Section 6.2) requires layered transparency across the training pipeline. Developing standardized, machine-verifiable disclosure formats would enable systematic AI corrigibility assessment.
6. **Legal doctrine integration.** Constitutional courts increasingly review digital infrastructure. How should such courts operationalize corrigibility? Integration with proportionality analysis, due process doctrine, and administrative law would extend the framework’s applicability.

7. **Network effect formalization.** A formal model of network-effect capture, specifying when a technically forkable system becomes practically unforkable, would sharpen the FORK test’s application to platform infrastructure.
8. **Agentic system requirements.** Section 8.4.1 sketches how AI agents require corrigible infrastructure. Specifying precise API contracts for agent-compatible infrastructure would translate corrigibility requirements into engineering specifications.

Acknowledgments

This framework was developed through analysis of India Stack and subsequent generalization to universal criteria. The author thanks the digital rights community for ongoing critique and refinement.

License

This work is released under CC0 1.0 Universal (Public Domain). Use, adapt, translate, and redistribute freely. No attribution required.

Data and Code Availability

The complete framework, including JSON Schemas for machine-readable disclosure and assessment, is available at:

- Framework documentation: <https://indiastack.in/dpi/>
- Infrastructure manifest schema: <https://indiastack.in/dpi/schema/infrastructure.json>
- Corrigibility assessment schema: <https://indiastack.in/dpi/schema/corrigibility.json>
- Paper source and repository: <https://github.com/AnivarAravind/corrigibility-framework>

A Architecture of Accountability

To move beyond qualitative debate, this framework establishes a formal logic for accountability. The machine-readable artifacts described below serve not merely as data formats, but as a *digital constitutional binding* that separates operator claims from auditor verification. By enforcing specific data structures, we convert policy rhetoric into falsifiable assertions of system state.

A.1 Logic: The Adversarial Manifests

The architecture relies on a separation of concerns mirroring the adversarial legal process. It employs two distinct document types (see Figure 7):

1. **infrastructure.json (The Operator’s Affidavit):** A signed declaration by the system operator asserting specific operational properties, governance commitments, and functional limitations.
2. **corrigibility.json (The Auditor’s Finding):** An independent, cryptographically signed evaluation that tests the operator’s claims against observable reality.

This architecture makes contradictions machine-readable. If an operator claims a governance mechanism is "binding" in their affidavit, but the auditor’s observation marks the `user_authority` field as "advisory," the mismatch serves as mathematically verifiable proof of governance failure.

A.2 Interpretation: The Non-Authoritativeness of Determination

A critical logic gate in this framework is the derivation of status.

Claim 6. *The determination object in `corrigibility.json` is a derived convenience field. It is **not authoritative**.*

Any value asserted in `determination` MUST be mechanically recomputed from the `tests` object by evaluators or courts. If an auditor asserts `corrigible: true` but the `tests` object contains a failure, the manifest is strictly invalid. This rule ensures that no actor (operator, auditor, or regulator) can "declare" a system corrigible by fiat; status exists only as the computed sum of observable structural properties.

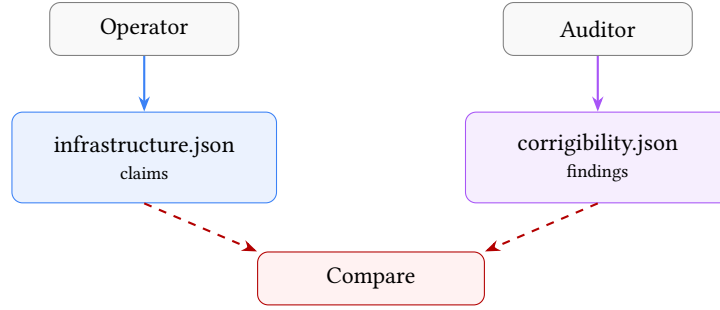


Figure 7: Adversarial Verification Architecture. Operators declare; Auditors verify. The structural gap constitutes the audit finding.

B Formal Structural Proofs

To formalize the assertion that “partial corrigibility is a fatal structural failure,” we model Digital Public Infrastructure as a standard feedback control system subject to environmental disturbance.

B.1 Proof of Open-Loop Instability (The “Fatal Failure” Theorem)

Let a DPI system state $y(t)$ (actual impact) be expected to track a reference target $r(t)$ (intended policy/rights), subject to environmental disturbance $d(t)$ (attacks, changing demographics, fraud).

The error signal (grievance) is defined as:

$$e(t) = r(t) - y(t) \quad (20)$$

In a closed-loop system, the corrective action $u(t)$ (GOVERN) is a function of the detected error via a feedback controller K (GOVERN logic) and a sensor transfer function H (AUDIT/EXIT mechanism):

$$u(t) = K(t) \cdot H(e(t)) \quad (21)$$

The plant dynamics (DPI operation) are given by:

$$\dot{y}(t) = u(t) + d(t) \quad (22)$$

Theorem 2 (Null-Feedback Instability). *If any component of the feedback path is structurally broken, specifically if Sensor Transparency $H = 0$ (Failed AUDIT) or Actuator Coupling $K = 0$ (Failed GOVERN), the system effectively operates as Open Loop. Under Open Loop conditions with non-zero integrating disturbance $d(t)$, the error $e(t)$ diverges unboundedly.*

Proof. If tests **AUDIT** or **EXIT** fail, $H = 0$. If test **GOVERN** fails, $K = 0$. In either case, the effective corrective feedback is $u_{fb} = 0$. The system evolution depends only on initial intent u_{open} and disturbance:

$$y(t) = \int_0^t (u_{open}(\tau) + d(\tau)) d\tau \quad (23)$$

Given that environmental variety $d(t)$ is stochastic and non-zero (Ashby’s Law), and without negative feedback to compensate:

$$\lim_{t \rightarrow \infty} |r(t) - y(t)| \rightarrow \infty \quad (24)$$

Thus, stability is impossible without the closure of the feedback loop. “Partial” corrigibility (where $K = 0$ or $H = 0$) yields the same asymptotic stability profile as total authoritarianism. $\square$

Corollary 2. *Corrigibility is a binary structural property. If any one of the five tests (EXIT, CODE, AUDIT, GOVERN, FORK) fails, the system is incorrigible.*

Justification: Each test corresponds to a necessary component of the feedback loop. Partial compliance does not preserve loop closure. This follows directly from Theorem 1: since $L(s) = K(s) \cdot P(s) \cdot H(s)$, if any multiplicative term is zero, the entire loop gain vanishes.

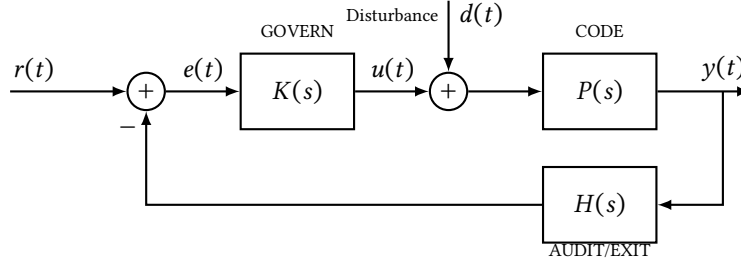


Figure 8: **Control-theoretic model of corrigible infrastructure.** $K(s)$: governance controller. $H(s)$: sensor/audit function. $P(s)$: plant (infrastructure). $d(t)$: environmental disturbance.

The closed-loop transfer function from reference to output is:

$$\frac{Y(s)}{R(s)} = \frac{K(s)P(s)}{1 + K(s)P(s)H(s)} \quad (25)$$

The transfer function from disturbance to output is:

$$\frac{Y(s)}{D(s)} = \frac{P(s)}{1 + K(s)P(s)H(s)} \quad (26)$$

Corrigibility requires that each component exists and is non-degenerate:

- EXIT $\Rightarrow e(t) \neq 0$ observable (error signal exists)
- AUDIT $\Rightarrow H(s) \neq 0$ (sensor functional)
- GOVERN $\Rightarrow K(s) \neq 0$ (actuator functional)
- CODE $\Rightarrow P(s)$ is intelligible
- FORK $\Rightarrow P, K, H$ replaceable if convergence fails

Active Deception as Sensor Attack. The pathologies defined in Section 5.3.2 (open-washing, safety-washing, audit theatre) fundamentally corrupt the sensor function $H(s)$: they either suppress the true error signal $e(t)$ or inject artificial stability metrics, thereby blinding the controller $K(s)$ to the actual divergence $y(t) - r(t)$. In control-theoretic terms, Active Deception renders $H(s) \rightarrow 0$ not by removing the sensor, but by making its output uncorrelated with the true system state.

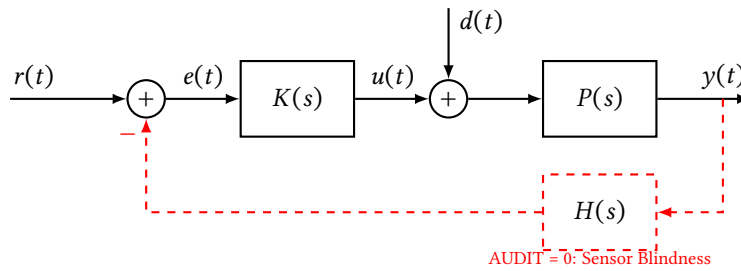


Figure 9: **Open-loop failure mode.** When AUDIT fails ($H = 0$), the feedback path is severed. Equivalent failure occurs if GOVERN ($K = 0$) or EXIT ($e(t) \equiv 0$) is removed.

B.2 The Strict Correction Velocity Inequality

For the system to remain stable not just asymptotically but locally (preventing irreversible harm), the correction rate must dominate the error rate. We formalize this with a friction coefficient.

Condition 1 (Strict Dynamic Stability). Let $\epsilon > 0$ represent the “friction cost” of correction (e.g., litigation time, protest cost). A system is stable iff:

$$\left| \frac{d}{dt} C(t) \right| \geq \left| \frac{d}{dt} E(t) \right| + \epsilon \quad (27)$$

Implication: Post-hoc remedies (Courts/Ombudsmen) have a linear or constant rate of operation ($\frac{dC}{dt} = k$). Digital errors propagate at exponential or high-frequency rates ($\frac{dE}{dt} \propto e^t$). Since there is no t for which a linear function dominates an exponential one, relying on external courts guarantees eventual system collapse. The correction logic must be intrinsic to the loop.

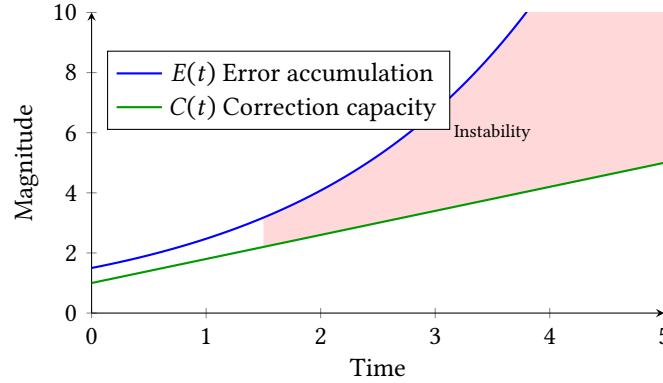


Figure 10: **Correction velocity mismatch.** Error $E(t)$ accumulates faster than correction $C(t)$ in bureaucratic systems. The shaded region represents accumulated unresolved harm.

B.3 Mapping to the Five Tests

Table 14: Mapping corrigibility tests to control-theoretic components

Test	Control Component	Failure Mode
EXIT	Error Signal	Reference collapse
CODE	Signal Integrity	Noise indistinguishable from intent
AUDIT	Sensor	Blind control
GOVERN	Actuator	No corrective force
FORK	Variety Expansion	Monolithic collapse

B.4 Agentic Interoperability Requirements

For an autonomous agent $\mathcal{A}$ to operate reliably within infrastructure $\mathcal{I}$, the following predicates must be strictly True:

1. **Reversibility:** $\forall$ state $s_t \in \mathcal{S}$, $\exists$ transition s_{t+1} (via EXIT) such that $\text{Cost}(s_{t+1}) < \infty$. (No deadlocks).
2. **Decidability:** $\forall$ input x , $\text{Compute}(x) \rightarrow \{0, 1\}$ is inspectable via CODE. (No hidden constraints).

Failure of these predicates renders $\mathcal{I}$ a “Hazmat Environment” for automated agents, effectively blocking the agentic economy.

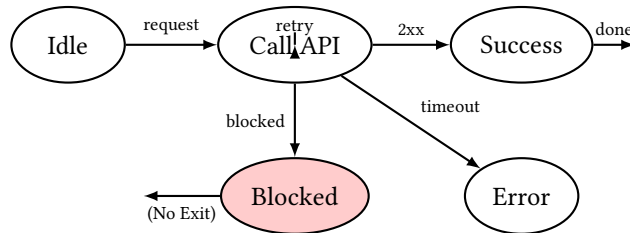


Figure 11: **Agentic automaton.** Without EXIT and observable error codes, agent loops indefinitely, times out, or escalates incorrectly.

B.5 Measurement Procedures for Reproducibility

To enable third-party reproduction of pass/fail determinations, each test requires specific evidence types:

Table 15: Evidence requirements for reproducible assessment

Test	Required Evidence	Verification Method
EXIT	(1) Legal mandate status, (2) documented alternative pathways, (3) penalty documentation	Survey of legal instruments; cost comparison with alternatives; user testimony
CODE	(1) Source repository URL, (2) build reproducibility proof, (3) execution path coverage	Hash verification; deterministic build; static analysis of critical paths
AUDIT	(1) API documentation, (2) data access agreement, (3) historical error rate publication	Attempt independent measurement; verify data freshness; compare operator vs independent findings
GOVERN	(1) Charter/statute URL with hash, (2) governance action log, (3) enforcement history	Legal analysis; search for overruled operator decisions; verify binding mechanism
FORK	(1) License text, (2) technical artifact availability, (3) resource cost estimate, (4) successful fork examples	License audit; attempt reproduction; cost modeling; historical fork survey

Reproducibility Standard. An assessment is reproducible if an independent auditor, given only the evidence artifacts listed above, reaches the same pass/fail determination. Disagreements must trace to (a) threshold calibration differences or (b) missing evidence, not to ambiguity in the test definitions themselves. Continuous metrics and threshold protocols are provided in Tables 11 and 12 (Section 8.1).

B.6 Network Adoption Model for Fork Feasibility

Model adoption fraction $\alpha(t)$. A community-spawned fork succeeds if α crosses critical mass α^* . Logistic adoption with social influence:

$$\dot{\alpha} = \beta\alpha(1 - \alpha) + s(t) \quad (28)$$

where $s(t)$ is exogenous seeding. Fork feasibility requires parameters such that $\exists t$ with $\alpha(t) \geq \alpha^*$. Practical condition:

$$s_{\min} \geq \alpha^* - \alpha_0 \quad (29)$$

or subsidies that accelerate adoption rate β .

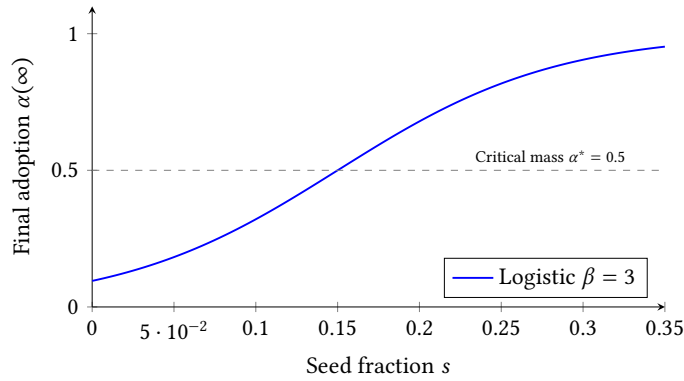


Figure 12: **Fork feasibility curve.** Below critical seeding, adoption does not reach critical mass. Parameters illustrative.

B.7 Extended Formal Models

B.7.1 Linearized Sensitivity Model

For small perturbations around a nominal operating point, the sensitivity function characterizes how disturbances propagate to the output:

$$S(s) = \frac{1}{1 + L(s)} \quad (30)$$

where $L(s) = K(s) \cdot P(s) \cdot H(s)$ is the loop gain. When any corrigibility test fails ($K = 0$, $H = 0$, or error signal blocked), $L(s) \rightarrow 0$ and $S(s) \rightarrow 1$: disturbances pass through unattenuated.

B.7.2 Discrete Integrator Example

Consider a discrete-time scalar integrator modeling cumulative harm:

$$y_{k+1} = y_k + d_k - u_k \quad (31)$$

where d_k is the disturbance (errors, attacks) and u_k is the correction. In open loop ($u_k = 0$):

$$y_N = y_0 + \sum_{k=0}^{N-1} d_k \quad (32)$$

For any non-zero mean disturbance $E[d_k] = \mu > 0$, the expected state grows linearly: $E[y_N] = y_0 + N\mu \rightarrow \infty$. This provides a simple, indisputable demonstration of open-loop divergence.

Friction cost in discrete model: The friction coefficient ϵ from Condition 1 manifests in this discrete model as a delay or reduction in the correction term. Specifically, if correction requires bureaucratic processing with latency τ time steps and efficiency loss $\eta < 1$, then:

$$u_k = \eta \cdot f(y_{k-\tau}, d_{k-\tau}) \quad (33)$$

where $f(\cdot)$ is the feedback policy. When $\tau > 0$ or $\eta < 1$, the effective correction rate is reduced, directly corresponding to the continuous-time friction cost $\epsilon > 0$. The stability condition $|dC/dt| \geq |dE/dt| + \epsilon$ translates to requiring the feedback policy to over-correct relative to the disturbance rate, accounting for these frictional losses.

B.7.3 Formal Error and Correction Rates

Let $E(t)$ denote cumulative error (total harm or divergence from policy) and $C(t)$ denote cumulative correction applied. Define:

- **Error generation rate:** $\dot{E}(t) = \frac{dE}{dt}$
- **Effective correction rate:** $\dot{C}(t) = \frac{dC}{dt}$

The system is stable only if the net error remains bounded:

$$E(t) - C(t) < M \quad \forall t > 0 \quad (34)$$

This requires $\dot{C}(t) \geq \dot{E}(t)$ on average. When correction operates through bureaucratic channels with bounded throughput $\dot{C}_{\max}$ while errors grow exponentially, there exists a critical time t^* beyond which correction can never catch up.

B.8 Summary of Formal Extensions

1. Linearized result ties removal of H , K , or EXIT to vanishing loop gain $L(s)$, exposing unstable process modes.
2. Integrator example supplies simple, indisputable divergence demonstration.
3. Correction velocity formalized via measurable $C(t)$ and $E(t)$.
4. Network model clarifies why legal forkability may fail in practice and yields policy levers (seeding, subsidies).
5. Agent automaton demonstrates need for explicit machine-facing EXIT/CODE/AUDIT primitives.

C Schema Specifications (v1.7)

The schemas below are condensed for publication. Full machine-readable JSON schemas with complete validation rules are available at the repository URLs listed in the Data and Code Availability section. Implementations must validate against the complete schemas to generate valid proofs of corrigibility.

C.1 Infrastructure Manifest (Operator Disclosure)

This schema forces operators to make definitive commitments. Note the specific inclusion of `offline_equivalent` to track the **EXIT** test and `when_uncertain` to track AI safety failures.

Listing 1: Operator Disclosure Schema (condensed)

```
{
  "$id": "https://indiastack.in/dpi/schema/infrastructure.json",
  "title": "DPI Operational Manifest v1.7",
  "required": ["schema_version", "meta", "access", "governance"],
  "properties": {
    "schema_version": {
      "const": "1.7",
      "description": "Schema version for forward compatibility"
    },
    "meta": {
      "required": ["system_id", "deployment", "lifecycle"],
      "properties": {
        "system_id": { "type": "string" },
        "deployment": { "enum": ["production", "staging", "pilot"] },
        "lifecycle": { "enum": ["active", "deprecated", "archived"] }
      }
    },
    "governance": {
      "required": ["model", "escalation_paths"],
      "properties": {
        "model": { "enum": ["community", "foundation", "corporate", "government", "closed"] },
        "escalation_paths": [{
          "required": ["level", "url", "binding"],
          "binding": { "type": "boolean" }
        }]
      }
    },
    "layers": {
      "type": "array",
      "description": "Architectural layers for decomposed evaluation",
      "items": {
        "required": ["layer_name", "controller", "transparency"],
        "properties": {
          "layer_name": { "type": "string" },
          "controller": { "type": "string" },
          "description": "Entity with governance authority",
          "transparency": { "enum": ["full", "partial", "none"] },
          "forkable": { "type": "boolean" }
        }
      }
    },
    "learned_components": [{
      "role": { "enum": ["advisory", "decision", "enforcement"] },
      "when_uncertain": { "enum": ["escalates", "fails_open", "fails_closed", "silent"] }
    }],
    "access": {
      "required": ["offline_equivalent"],
      "offline_equivalent": { "type": "boolean" }
    }
  }
}
```

C.2 Corrigibility Assessment (Auditor Assessment)

The auditor schema replaces subjective grading with hard observational measurements. Note the `penalty_ratio`, which quantifies the coerciveness of the system (the cost difference between digital and manual access).

Listing 2: Auditor Assessment Schema (condensed)

```
{
  "$id": "https://indiastack.in/dpi/schema/corrigibility.json",
  "title": "Corrigibility Assessment v1.7",
  "required": ["target", "assessed_by", "assessed_at", "tests", "determination"],
  "properties": {
    "target": { "type": "string" },
    "assessed_by": { "type": "string" },
    "assessed_at": { "type": "string", "format": "date-time" },
    "assessment_validity_period": {
```



```

    "type": "string", "format": "duration",
    "description": "ISO 8601 duration (e.g., P6M for 6 months)"
  },
  "tests": {
    "required": ["exit", "code", "audit", "govern", "fork"],
    "exit": { "pass": "boolean", "penalty_ratio": "number" },
    "code": { "pass": "boolean",
      "visibility": { "enum": ["full", "partial", "none"] } },
    "audit": { "pass": "boolean", "fix_latency_observed": "string" },
    "govern": { "pass": "boolean",
      "user_authority": { "enum": ["binding", "veto",
        "advisory", "none"] } },
    "fork": { "pass": "boolean", "barriers": ["string"] }
  },
  "layer_assessments": {
    "type": "array",
    "description": "Per-layer test results for decomposed evaluation",
    "items": {
      "required": ["layer_name", "tests"],
      "properties": {
        "layer_name": { "type": "string" },
        "tests": { "$ref": "#/properties/tests" }
      }
    }
  },
  "determination": {
    "description": "Derived outcome. MUST be recomputed by evaluators.",
    "required": ["corrigible", "tests_passed"],
    "corrigible": { "type": "boolean",
      "description": "TRUE iff tests_passed == 5" },
    "tests_passed": { "type": "integer", "minimum": 0, "maximum": 5 }
  }
}

```

D Verification and Enforcement

To distinguish valid governance from performative compliance, the framework enforces strict evidentiary rules for Identity and Authority.

D.1 Normative Rules

Rule A.9: Operational Definition of Binding Authority To satisfy the GOVERN test, any escalation path claimed as "binding" in the Manifest MUST include: (i) a resolvable reference to the statute or charter (`binding_proof_url`), (ii) a cryptographic hash of that instrument to prevent silent amendment, and (iii) historical evidence of enforcement against the operator. Assertions of authority that rely on post-execution bureaucratic discretion (e.g., ombudsmen, "feedback forms") DO NOT satisfy this rule and must be marked `false`.

Rule A.10: Chain of Custody Auditor manifests must be canonically serialized (RFC 8785), hashed (SHA-256), and signed (Ed25519) using a valid W3C Decentralized Identifier. Unsigned assessments are treated as schema-invalid. This creates a permanent, falsifiable record of who certified a system.

D.2 Identity Assurance Model

To prevent "Identity Washing", where operators use ephemeral keys to rubber-stamp their own systems—the framework employs a Tiered Assurance Model. Verification dashboards should weigh assessments accordingly:

1. **Tier 1: Self-Attested (`did:key`):** Ephemeral identity. Suitable for whistleblowers, individual researchers, or rapid-response checks. *Status: Informational.*
2. **Tier 2: Domain-Verified (`did:web`):** Identity cryptographically bound to a specific DNS domain (e.g., `did:web:eff.org`). Attribution to known civil society actors. *Status: Verified.*
3. **Tier 3: Institution-Verified:** Identity anchored in transparency logs or institutional trust registries. *Status: Regulatory Standing.*

E Framework Evolution

This framework did not evolve through theoretical speculation; it hardened through adversarial engagement with live systems. Each version update represents the closure of a specific "loophole" used by operators to claim public legitimacy while retaining private control.

Version 1.0 (Definition)

Established the baseline thesis: Corrigibility is a structural precondition, distinct from feature sets. It defined the five tests to answer the foundational question: *What distinguishes public infrastructure from private monopoly?*

Version 1.2 (Assessment)

Closed the Self-Attestation Loophole. Early trials showed operators self-declaring success. This version separated the *Infrastructure Manifest* from the *Corrigibility Assessment*, establishing the adversarial verification model where publicness must be proven by independent parties.

Version 1.4 (Logic)

Closed the Ambiguity Loophole. Eliminated partial credit and "maturity models." The shift to strictly binary logic established that failure in any single dimension breaks the feedback loop required by Ashby's Law, rendering the system unstable.

Version 1.5 (Invariance)

Closed the Exceptionalism Loophole. Extended the framework to AI and decentralized ledgers, demonstrating that neither statistical opacity nor cryptographic immutability exempts a system from the requirements of Reversibility (EXIT) and Correction (GOVERN).

Version 1.6 (Constraint)

Closed the Consultation Loophole. Redefined GOVERN as non-overrideable internal constraint rather than advice or redress. Explicitly disqualified post-execution remedies and advisory bodies from constituting system governance.

Version 1.6.2 (Reproduction)

Closed the Substitution Loophole. Distinguished EXIT (market choice) from FORK (reproduction). Switching landlords is not forking infrastructure.

Version 1.7 (Layering)

Closed the DID-Washing Loophole. Added layer-decomposed evaluation requiring analysis at each architectural stratum. Systems claiming corrigibility through identifier decentralization while retaining centralized governance at higher layers are now explicitly diagnosed. Introduced the propagation rule: $T_{\text{system}} = \bigwedge T(L_i)$.

F Anticipated Objections and Responses

This appendix addresses the strongest objections to the framework, organized by the discipline from which they originate.

F.1 Control Theory Objections

Objection 1: Binary model ignores continuous feedback dynamics. "Real control systems have continuous loop gain. Partial observability and partial actuation still provide some regulation. Your binary pass/fail ignores systems that are 'mostly corrigible.'"

Response: The objection conflates physical description with normative threshold. We do not dispute that real systems exhibit continuous variation in feedback quality. The binary model is a *legitimacy threshold*, not a physical claim (Section 8). A feedback loop with 50% sensor accuracy transmits some signal; this does not mean the system deserves public trust. The determination function D (Equation in Section 8.1) makes explicit that continuous inputs feed a binary output. The threshold is where we draw the line for public legitimacy, separable from the physics.

Objection 2: Where is the Lyapunov stability proof? "You claim systems diverge without feedback. Show the Lyapunov function. Without rigorous stability analysis, this is hand-waving."

Response: Appendix B provides the formal treatment. Theorem 1 (Null-Feedback Instability) proves that when $H = 0$ or $K = 0$, the system operates open-loop and diverges under non-zero disturbance. The discrete integrator example (Section B.6.2) provides an elementary proof: $y_N = y_0 + \sum d_k \rightarrow \infty$ for any non-zero mean disturbance. A full Lyapunov treatment would require specifying the state space for a particular DPI implementation; the framework operates at the architectural level, where the key insight—open loops diverge—is mathematically trivial.

F.2 Political Science Objections

Objection 3: You ignore legitimate state interests. “Governments have valid reasons for mandatory systems: national security, public health, anti-fraud. Your framework would prohibit pandemic contact tracing.”

Response: The framework does not prohibit mandatory systems; it diagnoses their structural properties. Emergency necessity explains but does not justify incorrigibility (Corollary to Proposition 2). A pandemic contact-tracing system can still satisfy CODE (open source), AUDIT (independent testing), GOVERN (binding privacy constraints), and FORK (interoperable protocol). Only EXIT is constrained by mandatory participation. The question is whether the other four tests pass—not whether mandates are ever legitimate. Section 8 explicitly notes that corrigibility is compatible with strict, non-negotiable rules.

Objection 4: Proposition 2 proves your framework is useless. “You prove essential services can never be fully corrigible. This makes your framework irrelevant for the systems that matter most—identity, payments, healthcare.”

Response: The proposition identifies a structural constraint, not a reason to abandon analysis. The architectural responses (Section 7.6.3) show two paths: protected alternatives (guarantee non-digital equivalents) or federated implementation (no single provider essential). X-Road demonstrates 4/5 at population scale. The finding that fully corrigible essential services require architectural innovation is the framework’s central contribution. Diagnosing why current systems fail is prerequisite to designing better ones.

F.3 Empirical Objections

Objection 5: Your pass/fail determinations are subjective. “Two auditors could reach different conclusions. Without precise, reproducible metrics, this is opinion dressed as science.”

Response: Section B.5 specifies evidence requirements for each test. Table 15 lists required artifacts and verification methods. The reproducibility standard states: an assessment is reproducible if an independent auditor, given only the evidence artifacts, reaches the same determination. Disagreements must trace to threshold calibration or missing evidence, not definition ambiguity. The JSON schemas (Appendix C) enforce structured disclosure. Future work includes inter-rater reliability studies.

Objection 6: FORK is illusory for AI systems. “You admit compute costs make AI retraining impractical. If FORK always fails for frontier AI, your framework just says ‘all AI is incorrigible’—trivially true and useless.”

Response: Section 6.4.3 specifies minimum evidence for learned system forkability. The key distinction is *inference forkability* (running weights) vs *training forkability* (reproducing the model). Only training forkability satisfies FORK for governance purposes. The evidence requirements (Table 6) are achievable: published training code, data manifests, and compute cost estimates. The claim is not that all AI fails FORK, but that systems withholding these artifacts fail. OLMo, Pythia, and academic models demonstrate that FORK-compliant AI is possible; the question is whether frontier labs choose to comply.

F.4 Summary

The framework withstands scrutiny from control theory (binary is threshold, not physics), political science (mandates can coexist with corrigibility on other dimensions), and empirical methodology (reproducible metrics exist). The strongest residual objection—network effects making FORK practically impossible even when legally permitted—is acknowledged in Limitations and proposed for formal treatment in Future Work.

G Sections Suitable for Standalone Publication

The following sections from this paper are self-contained and suitable for adaptation as op-eds, policy briefs, or standalone essays:

Active Deception (Section 5.3.2) Defines safety-washing, open-washing, and audit theatre as sensor attacks on accountability infrastructure. Suitable for technology policy audiences.

Climate Washing (Section 7.4.2) Analyzes environmental rhetoric in DPI discourse. Suitable for climate and technology policy intersection.

Engaging “Regulate the Loop” (Section 8.4.7) Responds to the “regulate later” objection. Suitable for development economics and policy audiences.

The Distributional Question (Section 8.3.2) Frames corrigibility as a question of whose suffering counts. Suitable for social justice and equity audiences.

Utility Does Not Equal Accountability (Section 8.5) Clarifies that useful systems can still be structurally captured. Suitable for general technology audiences.

The DPI Trilemma (Section 7.5.1) Sovereignty + Scale + Neutrality constraint. Suitable for international development and governance audiences.

A condensed “core” version of this paper (omitting these sections) is available for venues with stricter length constraints.

References

- Ashby, W. Ross. *An Introduction to Cybernetics*. Chapman & Hall, 1956.
- Ostrom, Elinor. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.
- Stallman, Richard. *Free Software, Free Society: Selected Essays*. GNU Press, 2002.
- Winner, Langdon. Do Artifacts Have Politics? *Daedalus*, 109(1):121–136, 1980.
- Hirschman, Albert O. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*. Harvard University Press, 1970.
- Lessig, Lawrence. *Code: Version 2.0*. Basic Books, 2006.
- Bowker, Geoffrey C. and Star, Susan Leigh. *Sorting Things Out: Classification and Its Consequences*. MIT Press, 1999.
- G20. New Delhi Leaders’ Declaration. G20 Summit, September 2023.
- United Nations Development Programme. Digital Public Infrastructure: Definitions and Core Concepts. UNDP Digital Strategy Office, 2024.
- Gebru, Timnit, Morgenstern, Jamie, Vecchione, Briana, et al. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- Mitchell, Margaret, Wu, Simone, Zaldivar, Andrew, et al. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.
- Bhatia, Gautam. *The Transformative Constitution: A Radical Biography in Nine Acts*. HarperCollins India, 2019.
- Leveson, Nancy G. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- Khera, Reetika. *Dissent on Aadhaar: Big Data Meets Big Brother*. Orient BlackSwan, 2019.
- Cohen, Julie E. *Between Truth and Power: The Legal Constructions of Informational Capitalism*. Oxford University Press, 2019.
- Supreme Court of India. Justice K.S. Puttaswamy (Retd.) v. Union of India. Writ Petition (Civil) No. 494 of 2012, (2017) 10 SCC 1.
- Collingridge, David. *The Social Control of Technology*. Frances Pinter, 1980.
- Pasquale, Frank. *The Black Box Society: The Secret Algorithms That Control Money and Information*. Harvard University Press, 2015.
- World Wide Web Consortium. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, July 19, 2022. <https://www.w3.org/TR/did-core/>
- European Union. Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 (eIDAS 2.0). *Official Journal of the European Union*, 2024.
- Khera, Reetika. Impact of Aadhaar on Welfare Programmes. *Economic and Political Weekly*, 52(50):61–70, 2017.
- European Commission. Study about the Impact of Open Source Software and Hardware on Technological Independence, Competitiveness and Innovation in the EU Economy. European Commission, Directorate-General for Communications Networks, Content and Technology, 2021.